Task:

① Employee Management:

i) Create sql database

↓

Create table employee

↓

[emp_id int,
name varchar(100),
age int,
department_id int,
Phone int;]

→ insert into table { values }

[insert into employee values (emp_id, Name, age)]

↓

* We can work with sql table with orderby, groupby, where commands and using different aggregate function.
* We use commands like update, store, delete and retrive.
* We can use joins commands

② Connect the database its library using
* sudo apt update
* sudo apt install mysql-client-y
* mysql --version
* mysql -u your_username -p

③ AWS

AWS

↓ open instance

Create instance (give name)

↓

Generate key

↓

Key pair name

↓

Launch instance → click instance ID

↓

Task

* Connect with EC2 instance

↓

Security Group → inbound → change http as port 80 for live server

↓

In SSH client → we need to copy the public and connect to linux

↳ Look download .pem and paste SSH id and connect

* Linux command ⇒ wsl -l -v

→ pwd

→ ls

→ mkdir

* Create csv file

↓

Create IAM role → S3 Bucket → New folder → upload the csv file in that folder → Then create admin in amazon Redshift ⇒ which act as analytical report

↓

In Redshift open editor

↓

connect with database

↓

copy table

from 'S3 URN'

IAM_role

DELIMETER ','

IGNOREHEADER 1

REGION

↓

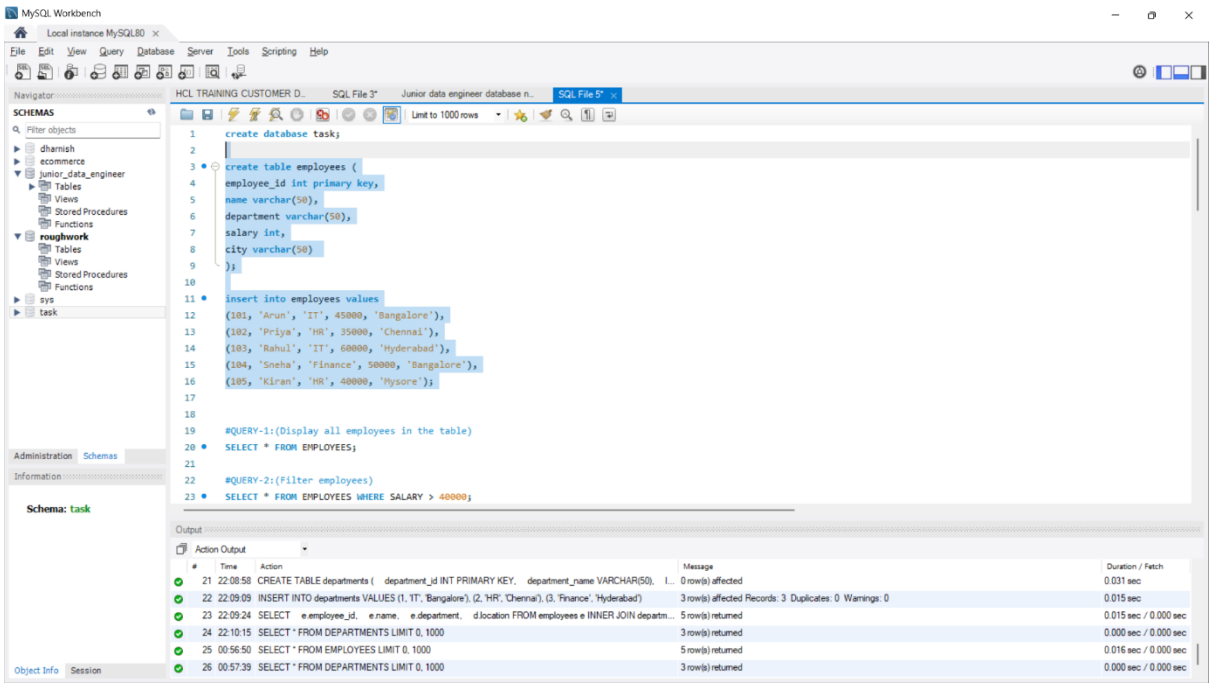
Select * from employee ⇒ There shows reports

OLAP

1. Creation of Database Tables

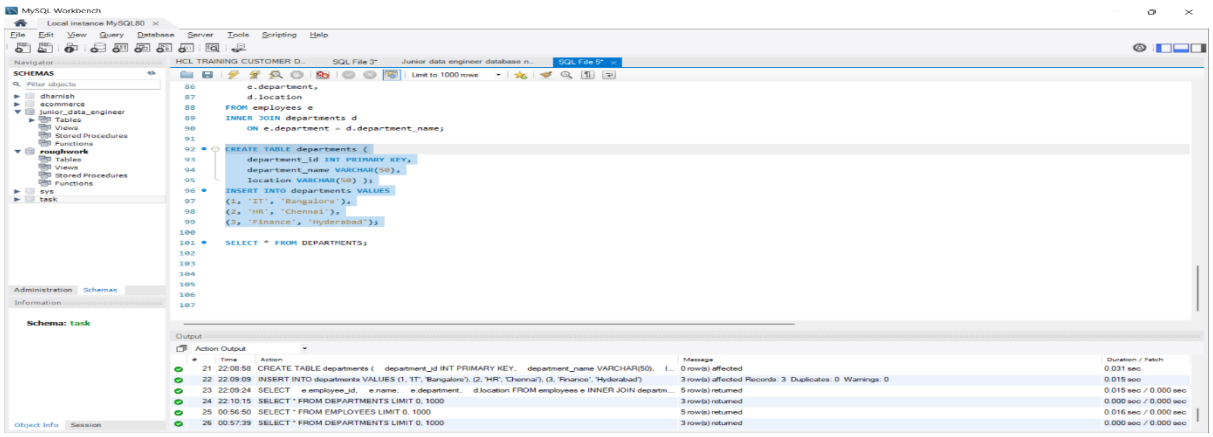
1.1 Creating the Employees Table

Description: The `employees` table stores employee information, including employee ID, name, department, salary, and city. It is used to practice SQL queries such as filtering, sorting, grouping, aggregate functions, and subqueries.



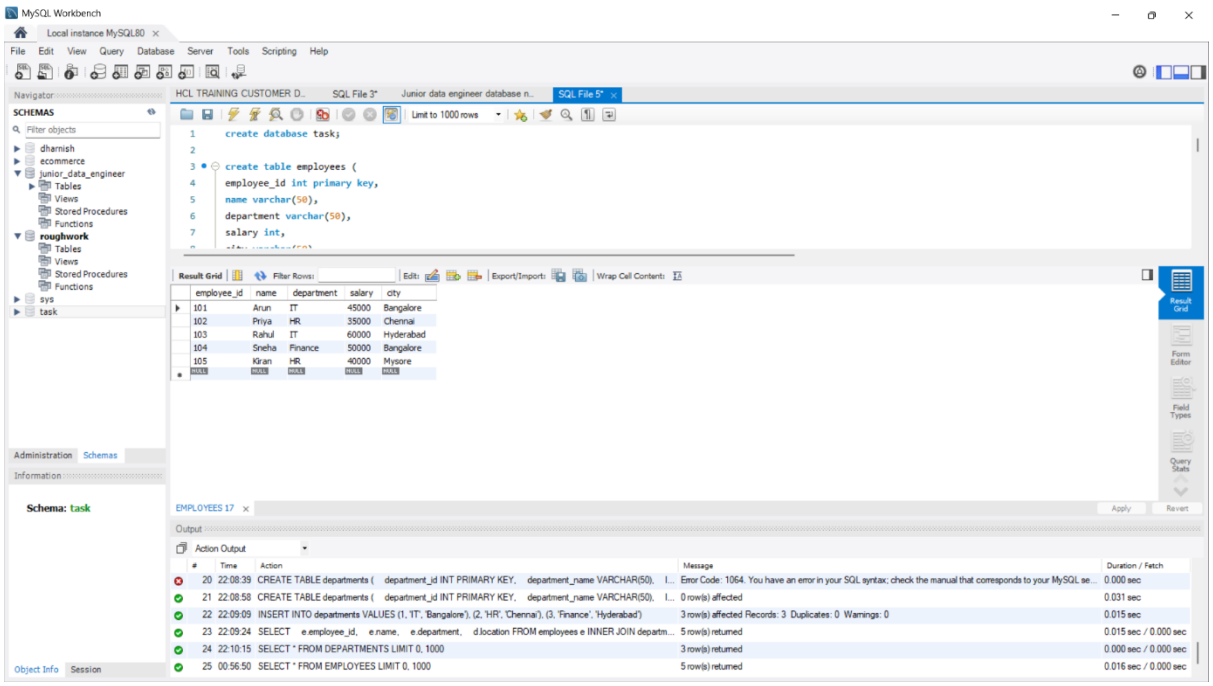
1.2 Creating the Departments Table

Description: The `departments` table stores department-related information, including department ID, department name, and location. It is used with the `employees` table to practice SQL joins and retrieve combined information.



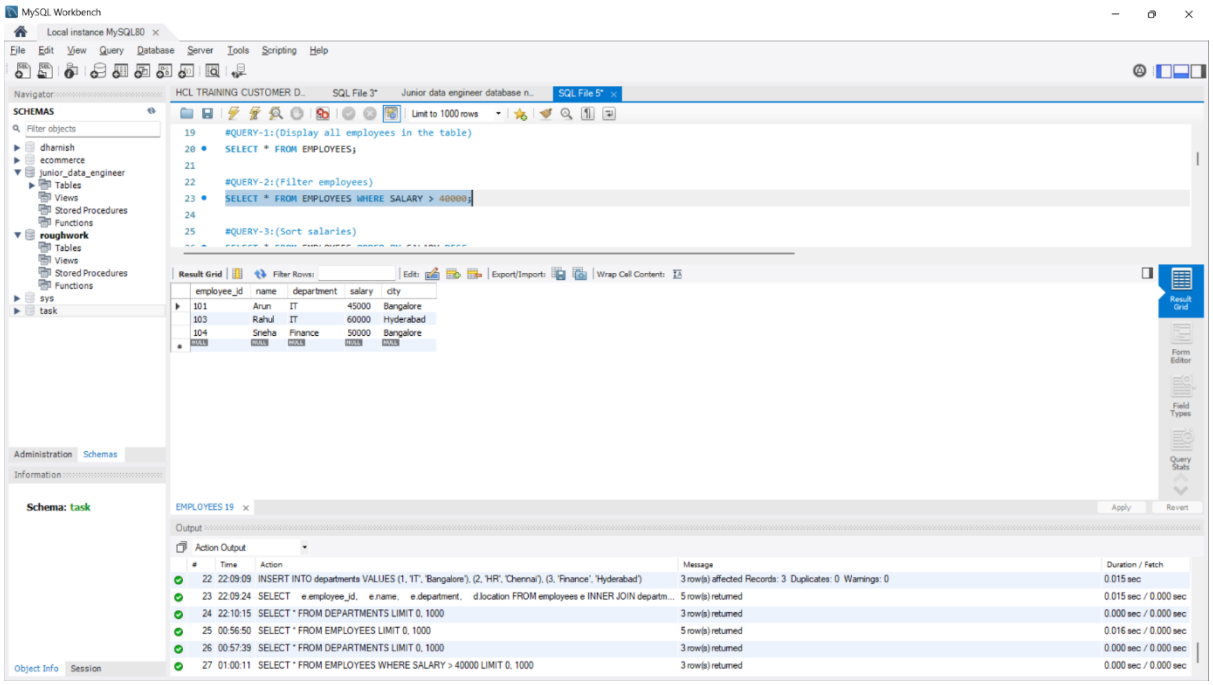
Query 1 — Display all employees

Objective: Retrieve all records from the employees table.



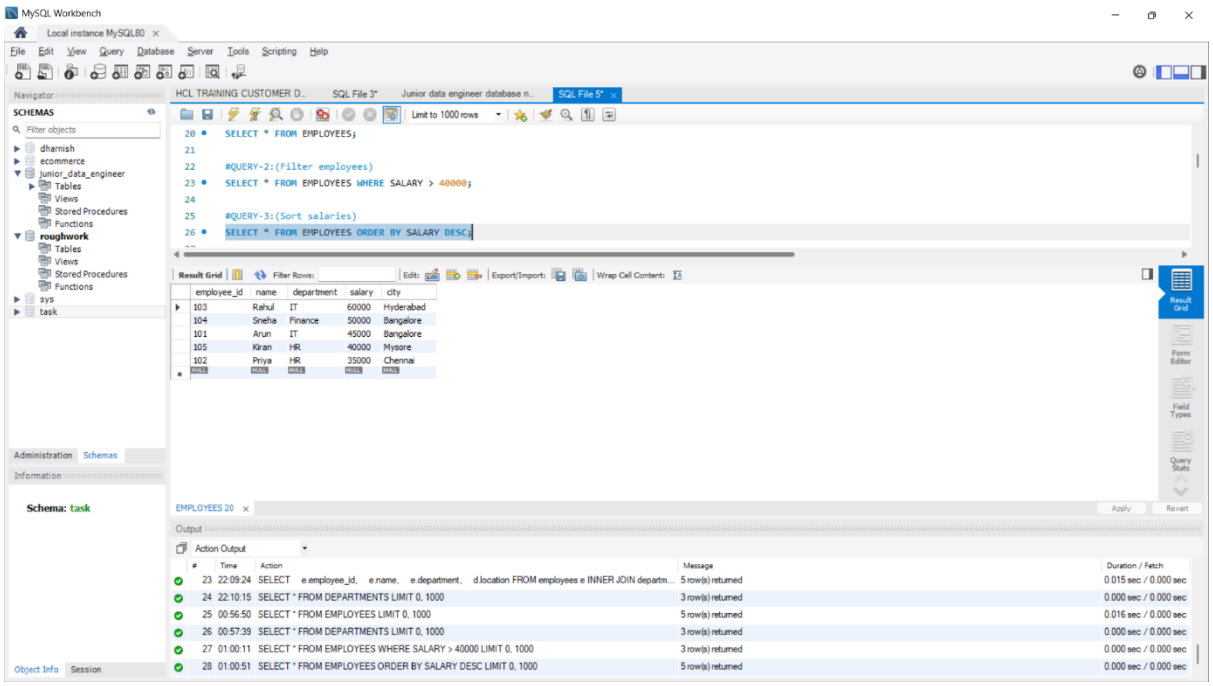
Query 2 — Filter employees by salary

Objective: Retrieve employees earning more than ₹40,000.



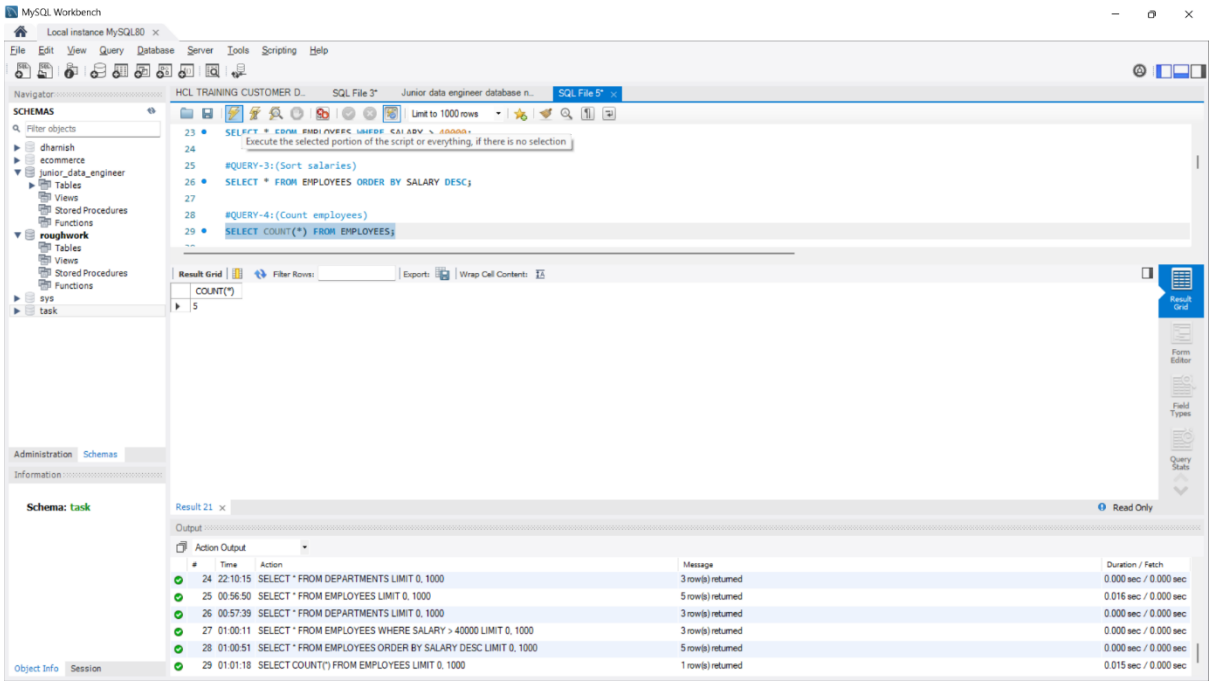
Query 3 — Sort employees by salary

Objective: Display employees in descending salary order.



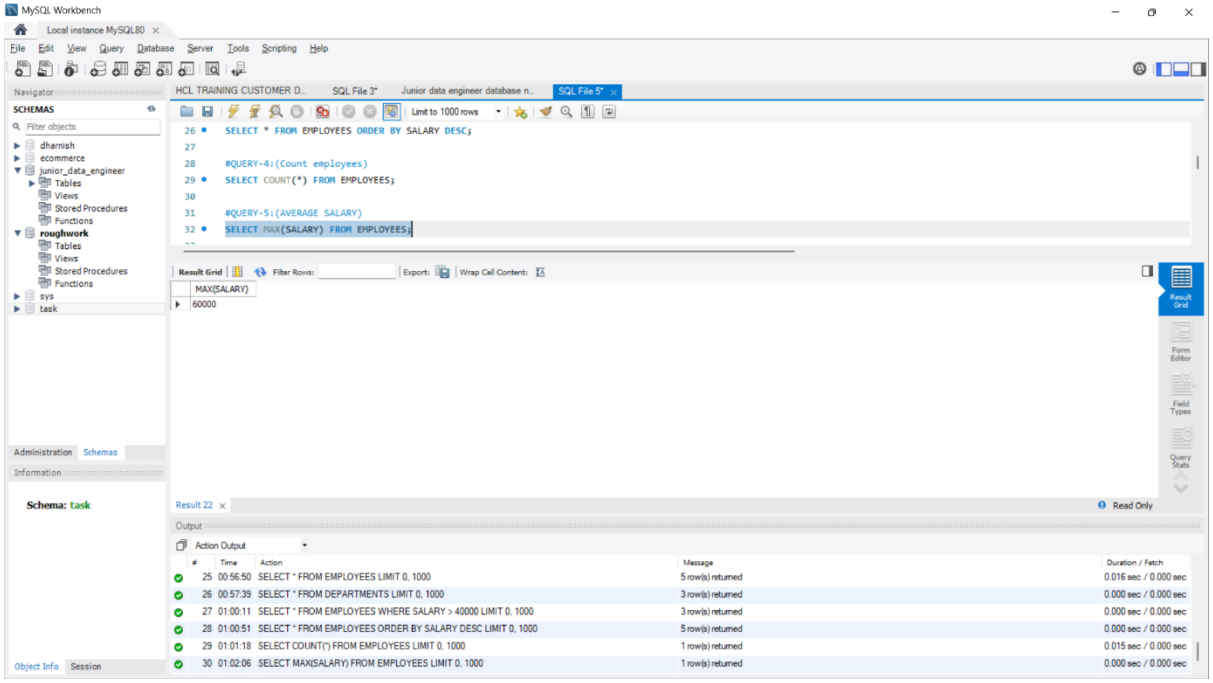
Query 4 — Count employees

Objective: Find the total number of employees.



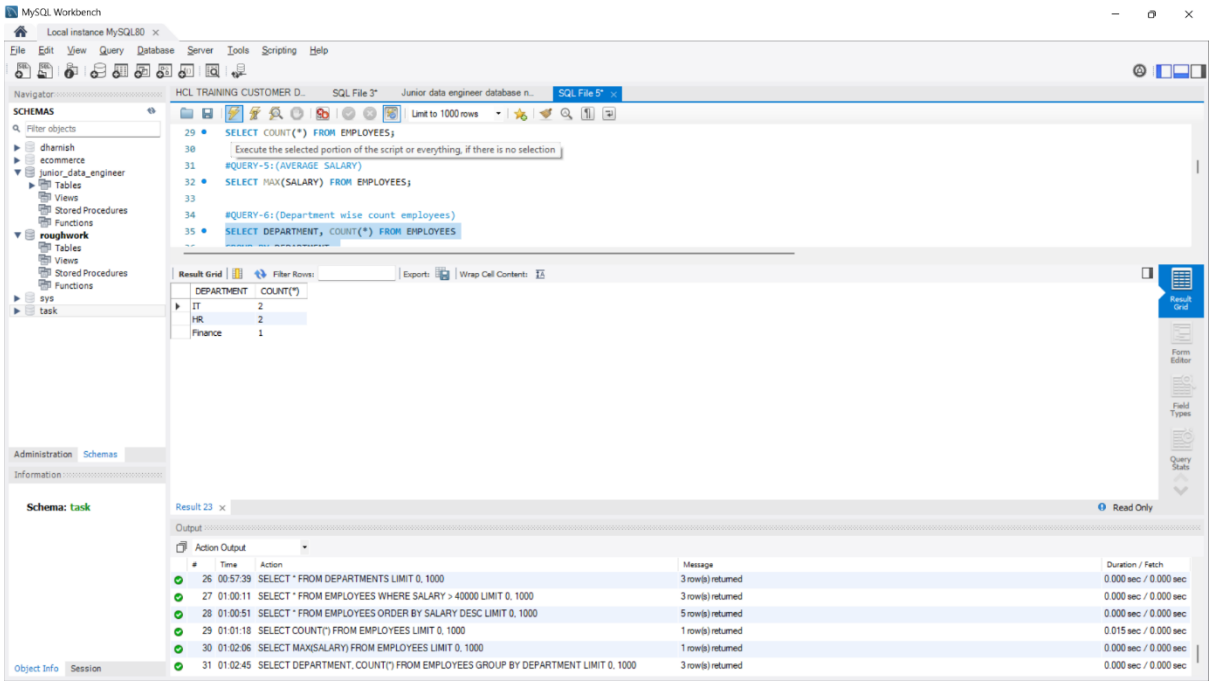
Query 5 — Find the maximum salary

Objective: Identify the highest salary in the employee table.



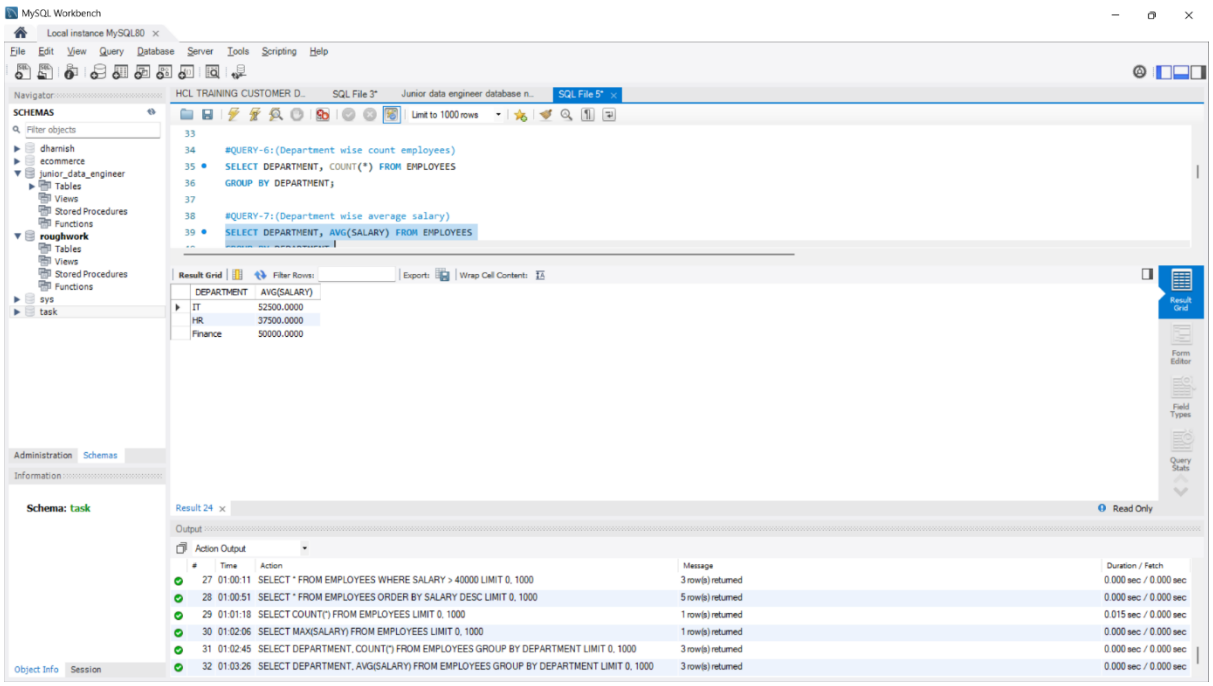
Query 6 — Count employees by department

Objective: Calculate the number of employees in each department.



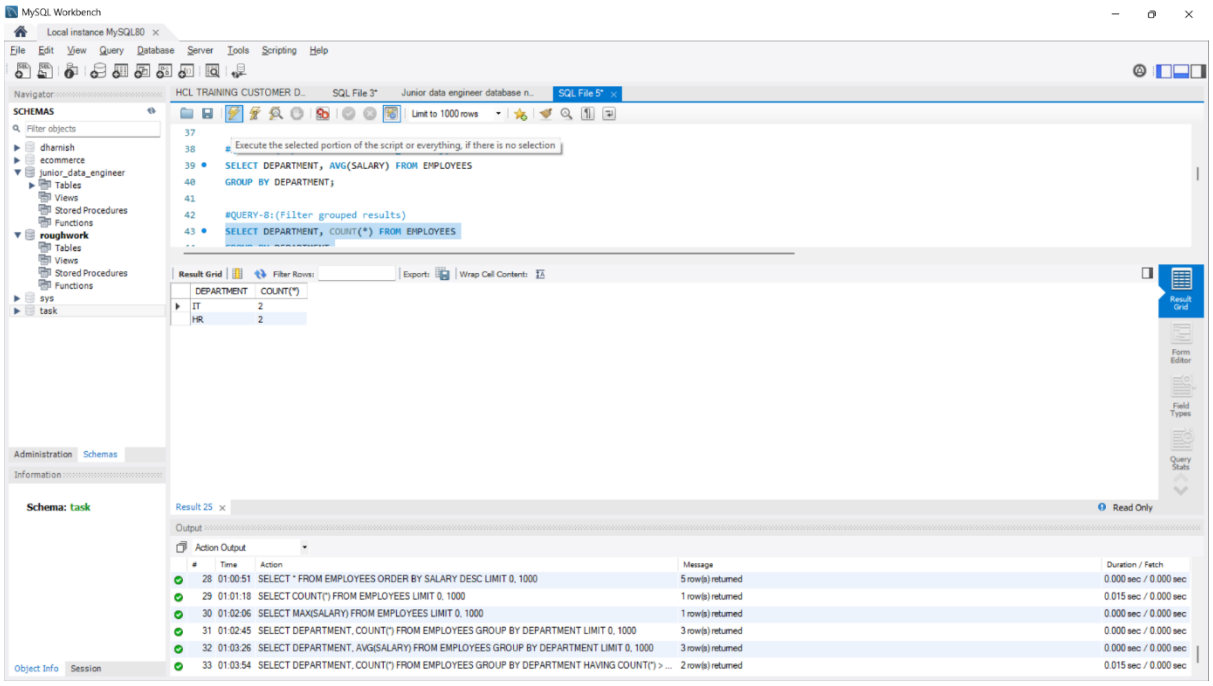
Query 7 — Calculate department-wise average salary

Objective: Find the average salary for each department.



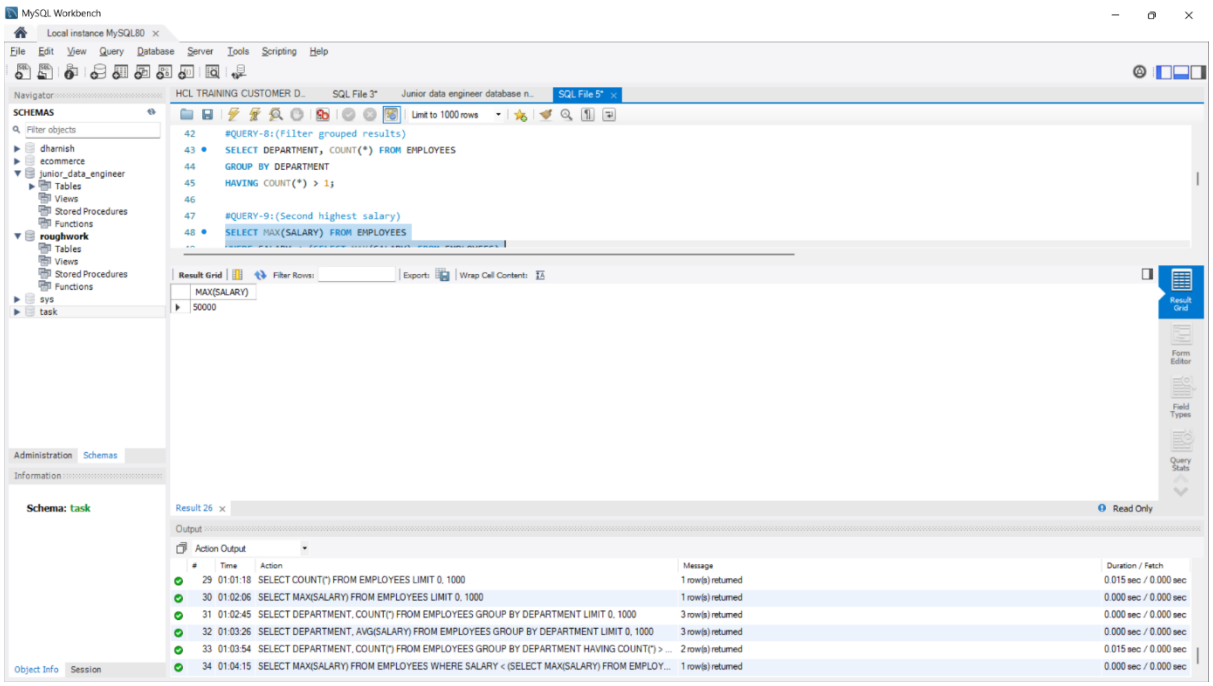
Query 8 — Filter grouped department results

Objective: Apply a condition to grouped department data.



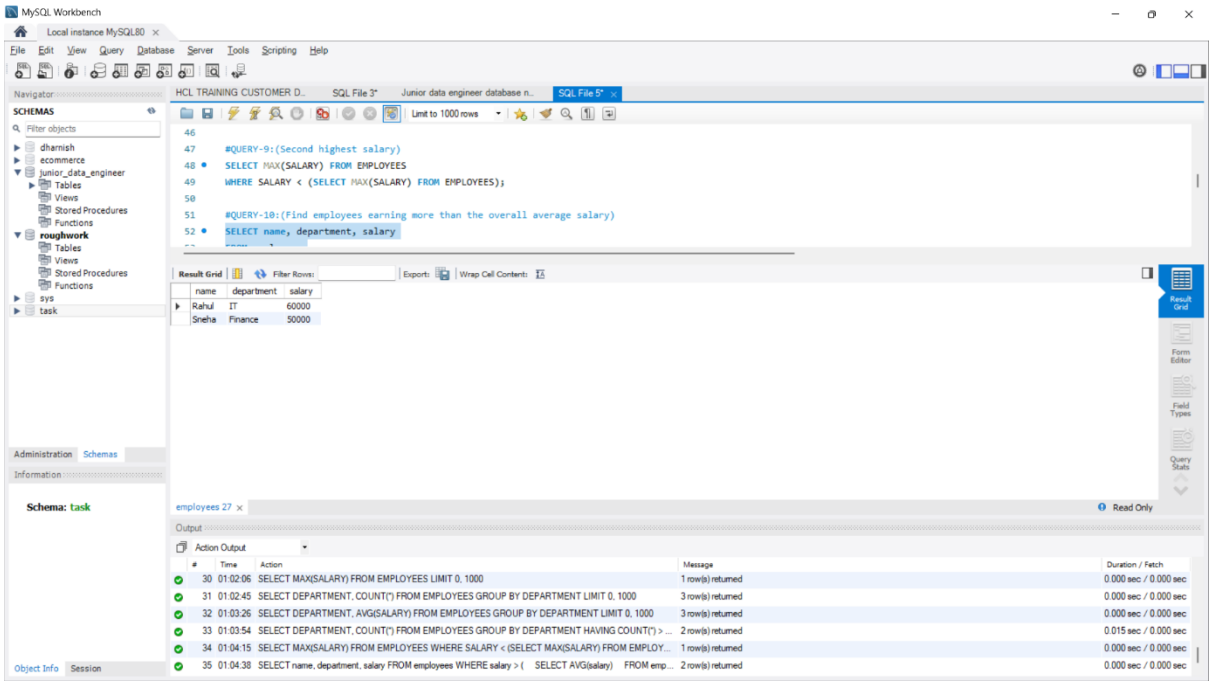
Query 9 — Find the second-highest salary

Objective: Retrieve the second-highest distinct salary.



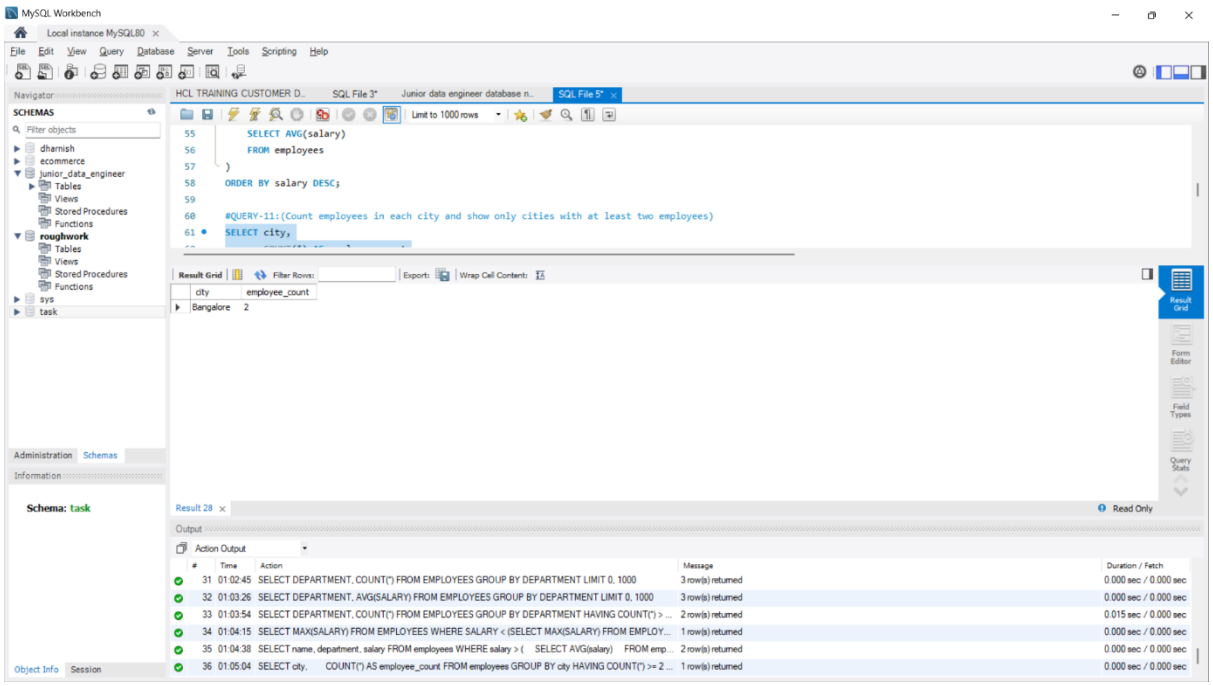
Query 10 — Find employees earning above average salary

Objective: Retrieve employees whose salaries are higher than the overall average.



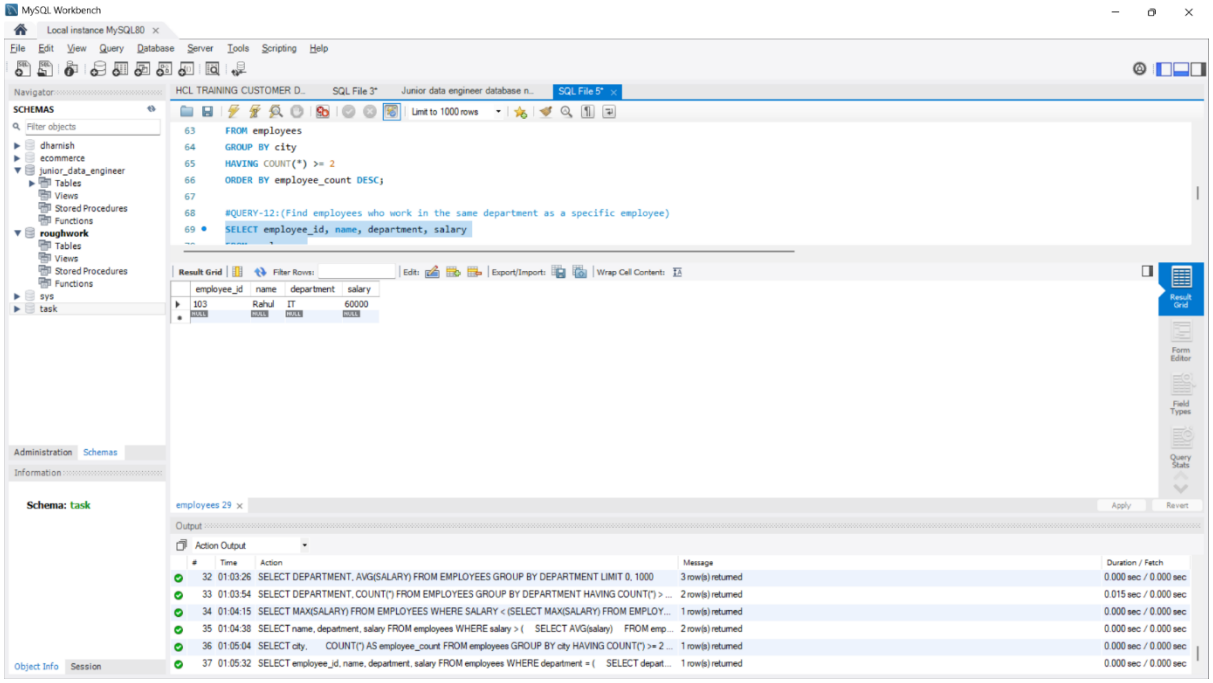
Query 11 — Count employees by city

Objective: Find cities with at least two employees.



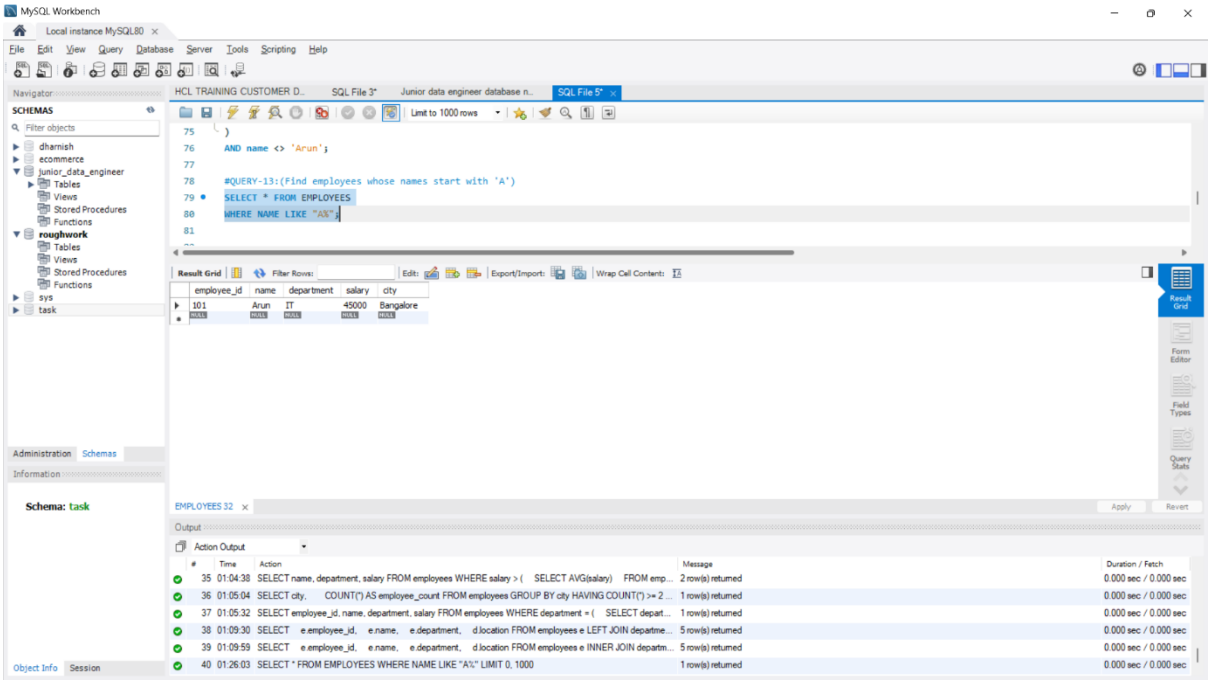
Query 12 — Find employees in the same department as a specific employee

Objective: Retrieve employees who belong to the same department as Arun, excluding Arun.



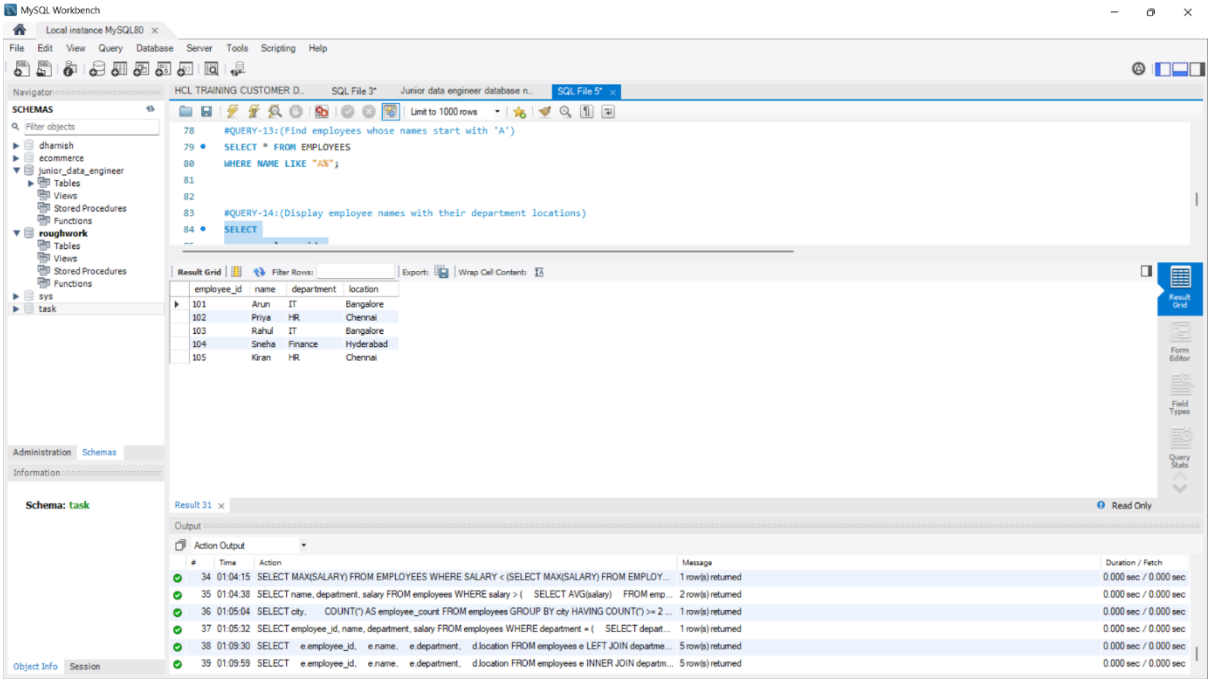
Query 13 — Find employees whose names start with A

Objective: Search for employees whose names begin with the letter A.



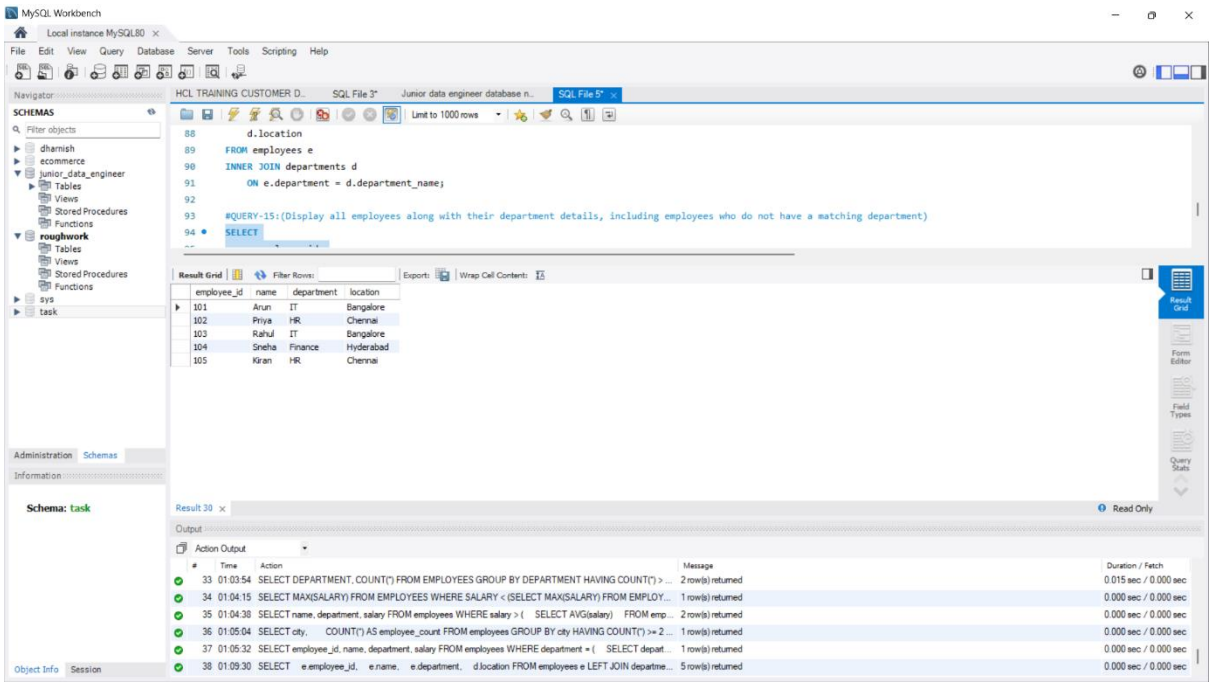
Query 14 — Display employees with department locations using INNER JOIN

Objective: Combine employee details with their department locations.



Query 15 — Display employees and department details using LEFT JOIN

Objective: Display all employees, including those without a matching department record.

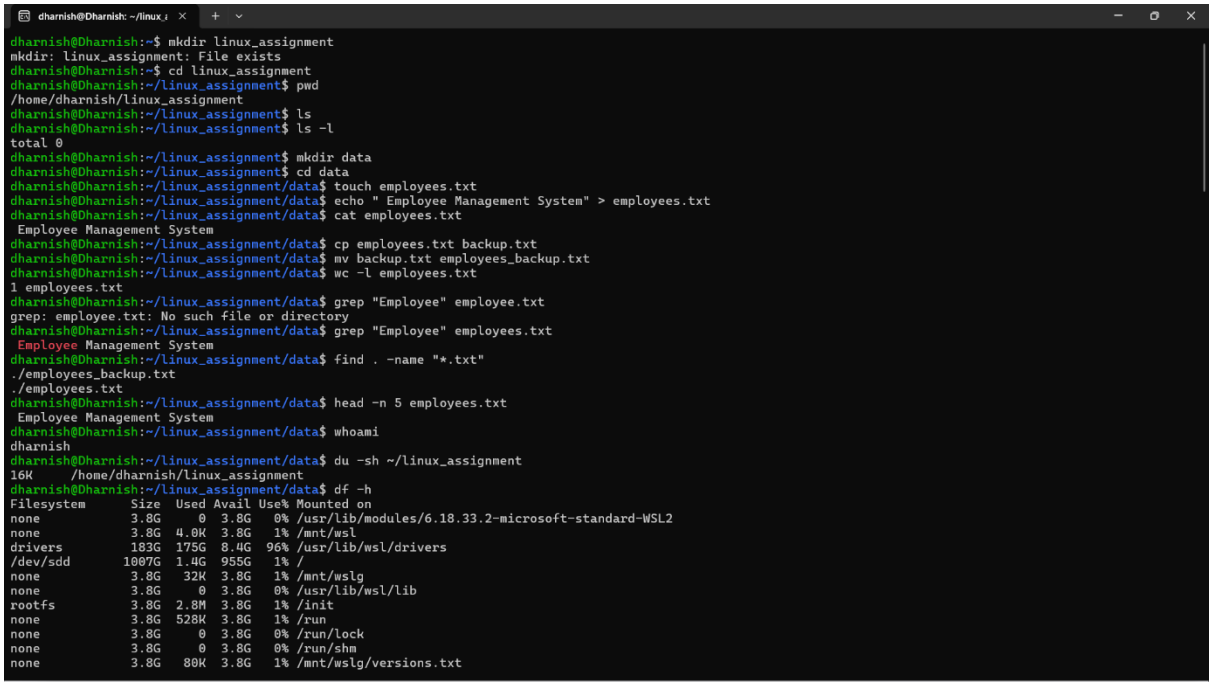


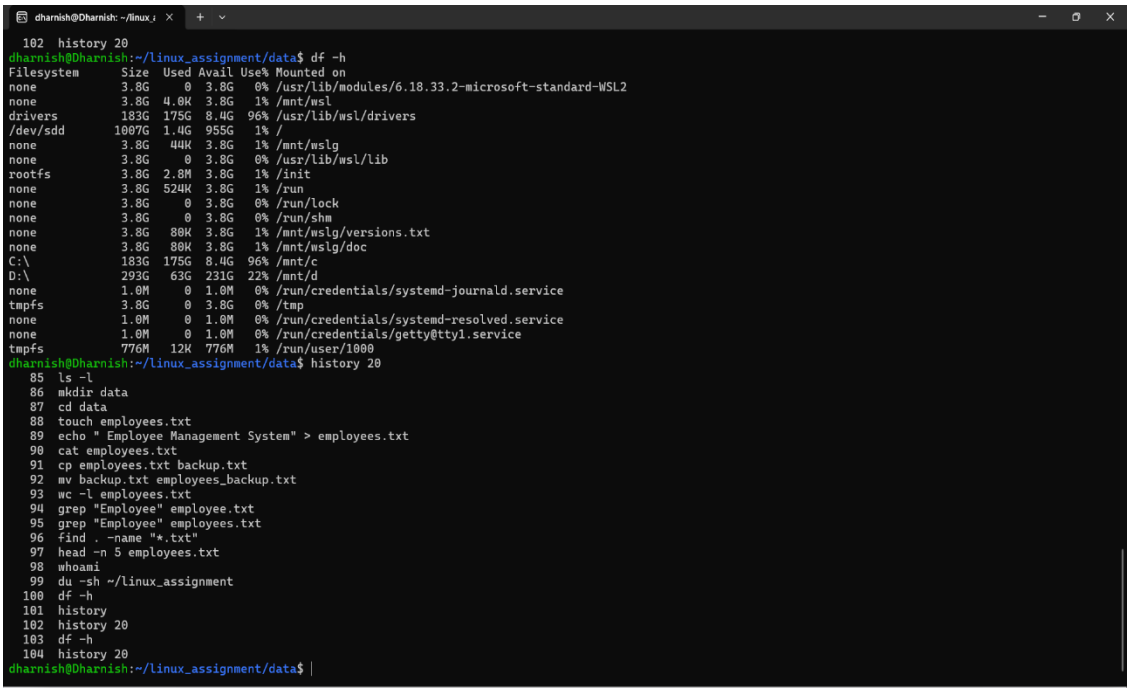
Conclusion: The SQL practice section covered data retrieval, filtering, sorting, aggregate functions, grouping, subqueries, pattern matching, and table joins. The queries were executed using MySQL Workbench, and the resulting outputs were captured as screenshots. This practice helped demonstrate how SQL can be used to retrieve and analyze employee information stored in relational database tables.

2.Introduction to Linux

Linux is an operating system widely used for server administration, cloud computing, and data engineering. The Linux command-line interface allows users to create directories, manage files, inspect data, search text, and monitor disk usage.

In this practical exercise, Linux commands were executed using Ubuntu through Windows Subsystem for Linux (WSL). A working directory named linux_assignment was created, and a data directory was used to perform file-management operations related to the Employee Management System.





Conclusion: The Linux practical exercise provided hands-on experience with directory management, file creation, copying, renaming, text searching, and filesystem inspection using Ubuntu through WSL. A project directory was created for the Employee Management System, and a text file was used to practice common file operations. Commands such as `grep`, `find`, `wc`, `du`, and `df` demonstrated how Linux supports basic data inspection and system administration tasks. These skills are useful when working with cloud servers and data engineering environments.

3.Introduction to AWS

Amazon Web Services (AWS) is a cloud computing platform that provides services for storing, processing, and analyzing data. In this project, Amazon S3 was used for data storage, AWS IAM for access management, AWS Glue for data discovery and transformation, and Amazon Athena for querying the output.

Project objective

The objective was to build a basic employee data engineering pipeline that takes a CSV file from Amazon S3, discovers its schema using AWS Glue, applies a transformation to remove the `city` field, and stores the resulting data in Parquet format in the curated S3 folder. Amazon Athena was then used to validate the generated output.

Overall architecture

Employee CSV

S3 — raw/

AWS Glue Crawler

Discovers schema and registers metadata

Glue Data Catalog

glue_training_db → employees

AWS Glue Visual ETL

Read source → Drop city → S3 target

Curated Parquet Output

S3 — curated/

Amazon Athena

Query and validate the output

This diagram summarizes the intended workflow.

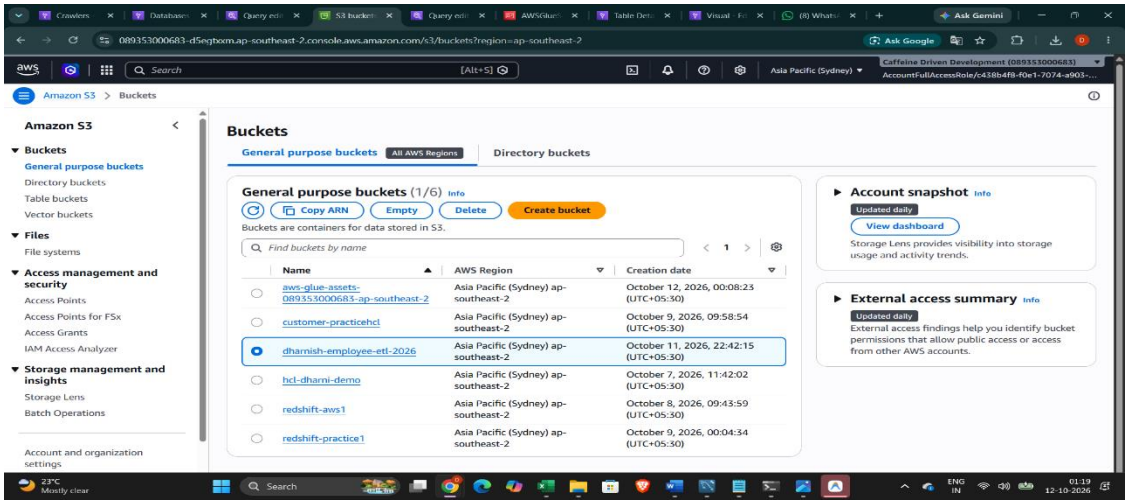
Create the Amazon S3 bucket and upload the raw dataset

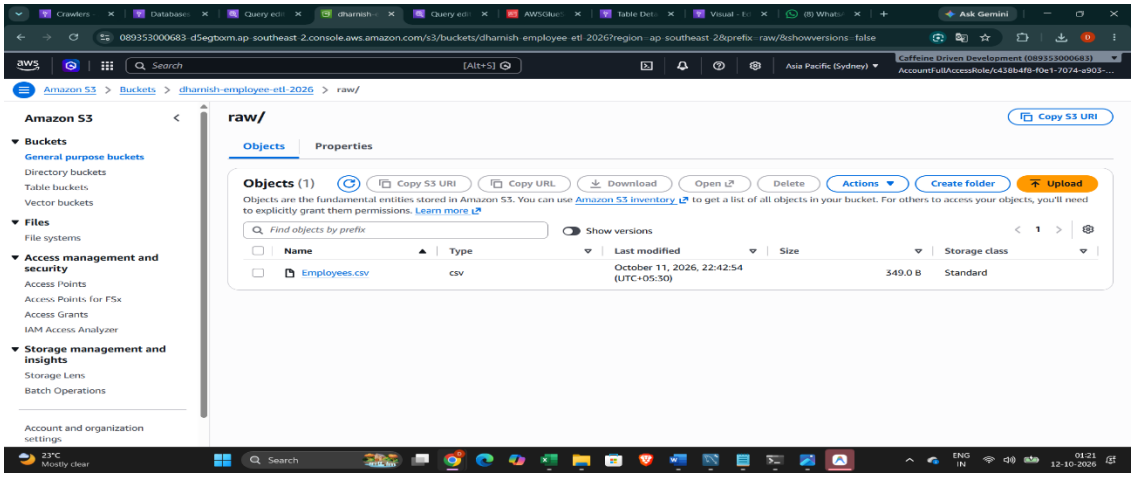
Objective: Store the original employee CSV file in Amazon S3.

Procedure:

1. Open the AWS Management Console and navigate to Amazon S3.
2. Create an S3 bucket using your actual bucket name.
3. Create a folder named `raw/`.
4. Upload `employees.csv` into the `raw/` folder.
5. Create a folder named `curated/` for the processed output.

Explanation: Amazon S3 acts as the storage layer for the data pipeline. The raw folder holds the original employee dataset, while the curated folder is the destination for transformed data.





Step 2 — Configure the IAM role

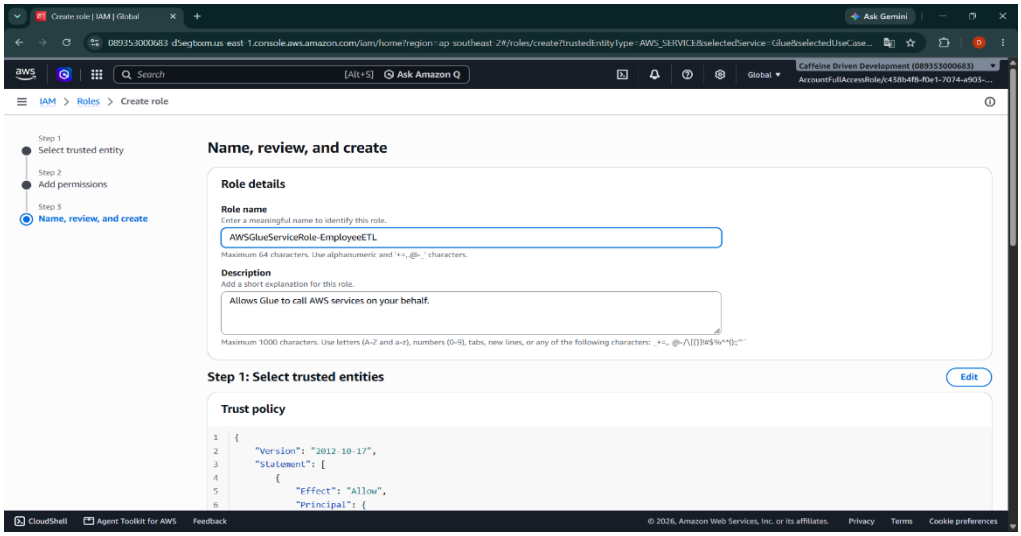
Objective: Give AWS Glue the permissions it needs to run the data processing workflow.

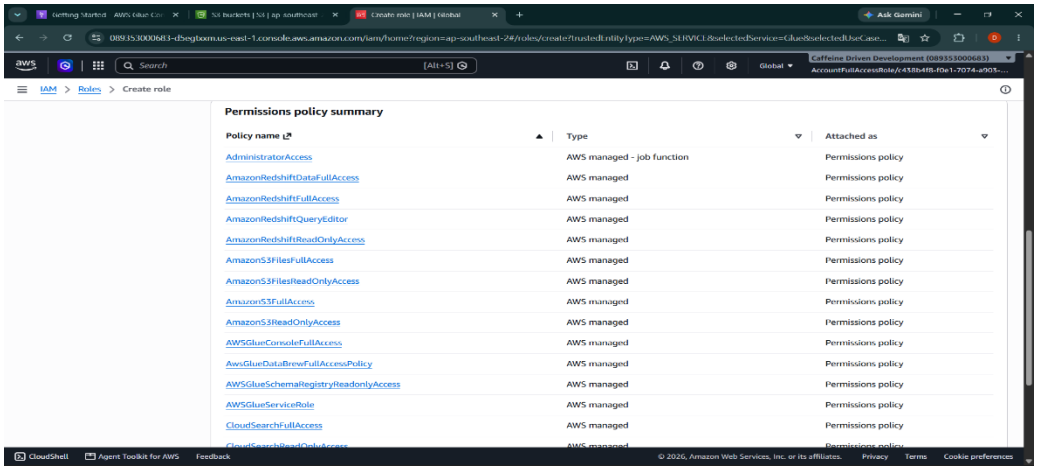
Procedure:

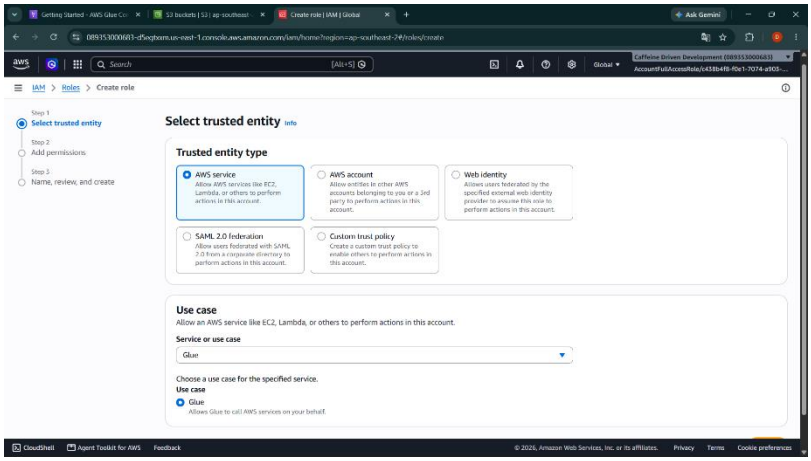
1. Open IAM in the AWS Console.
2. Create or select the role intended for AWS Glue.
3. Attach the required Glue service permissions and any additional permissions needed for the S3 source and target locations.
4. Select this role when configuring the crawler and ETL job.

Explanation: IAM controls which AWS resources a service can access. The Glue role allows the crawler and ETL job to perform their authorized operations without using personal credentials.

IAM ROLES AND PERMISSIONS







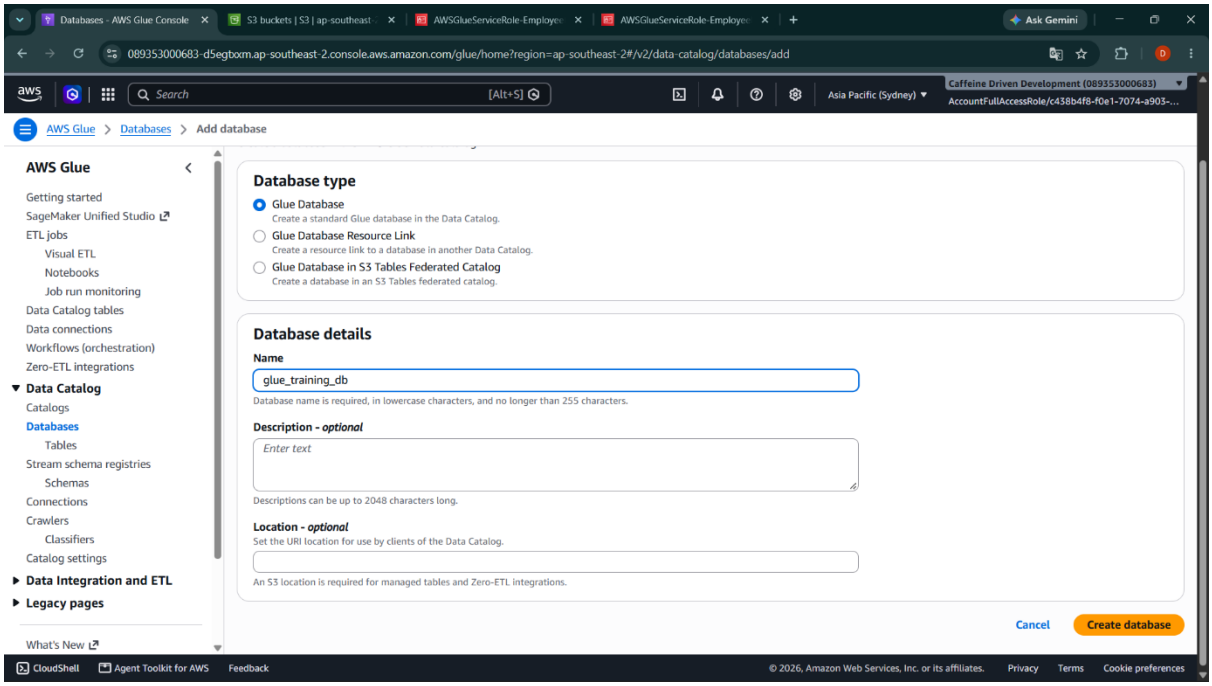
Step 3 — Create the AWS Glue database

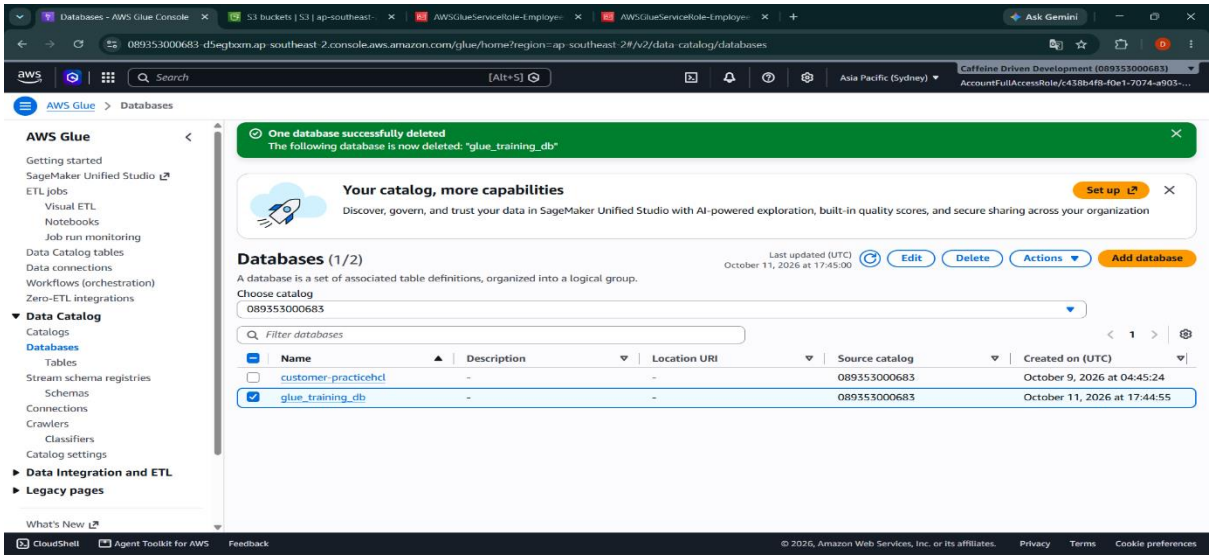
Objective: Create a metadata database for the employee dataset.

Procedure:

1. Open AWS Glue.
2. Navigate to Data Catalog → Databases.
3. Choose the option to add a database.
4. Enter `glue_training_db` as the database name.
5. Save the database and verify that it appears in the database list.

Explanation: The Glue Data Catalog stores metadata about datasets, including their tables, columns, and data types. The database provides a logical location for organizing the employee table.





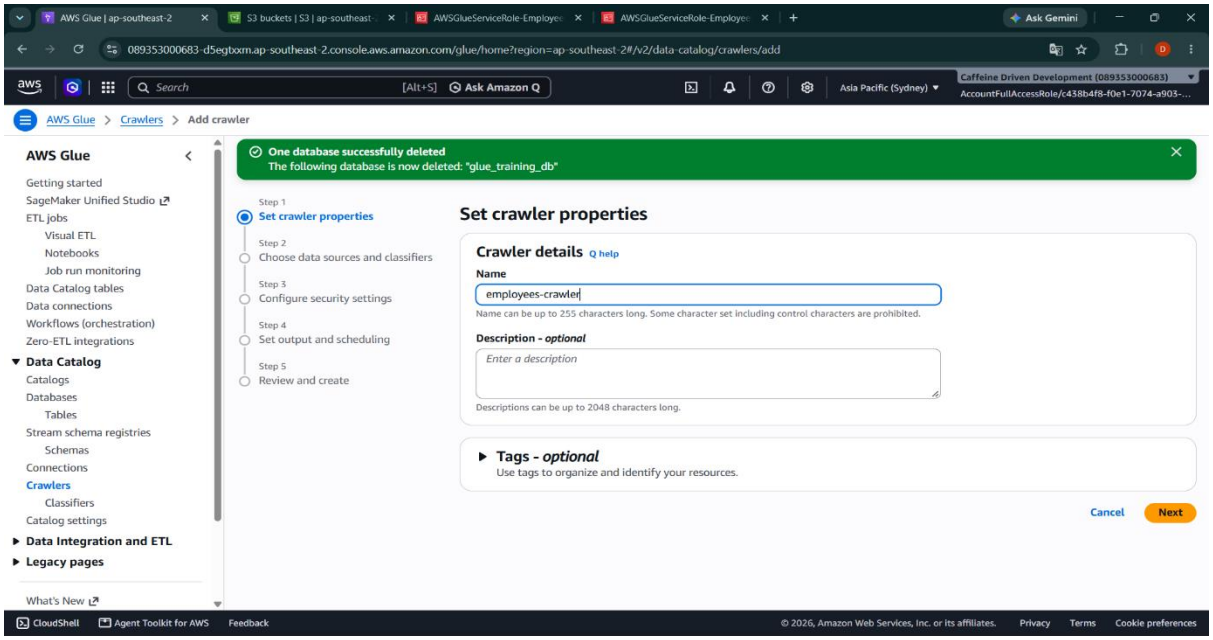
Step 4 — Create and configure the Glue Crawler

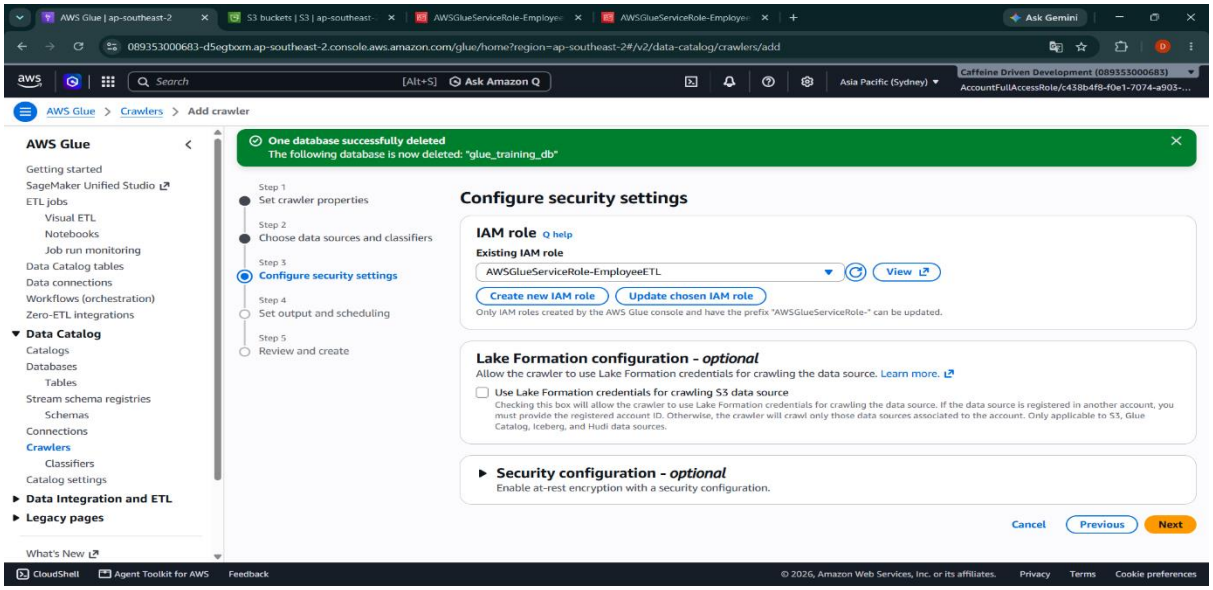
Objective: Discover the structure of `employees.csv` and register it in the Glue Data Catalog.

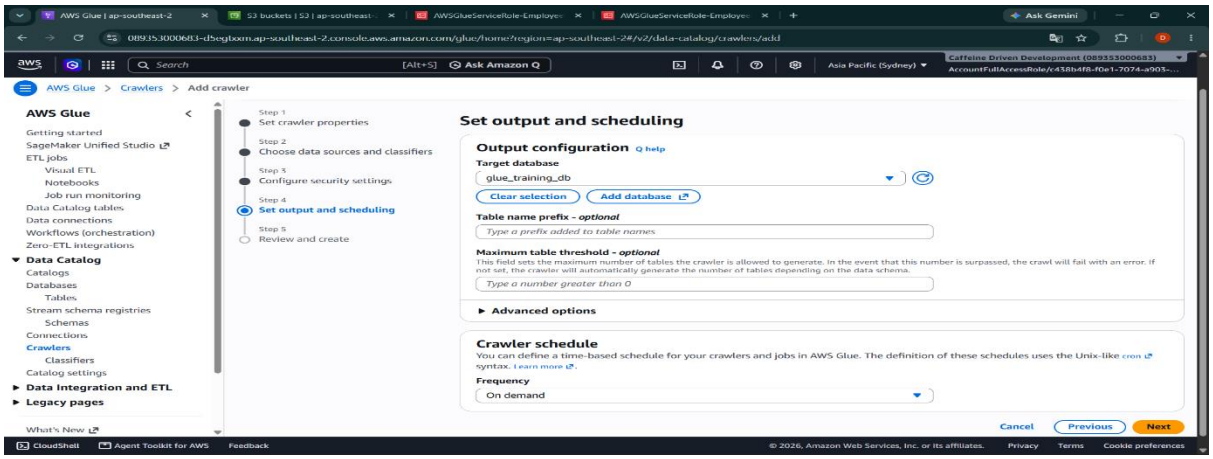
Procedure:

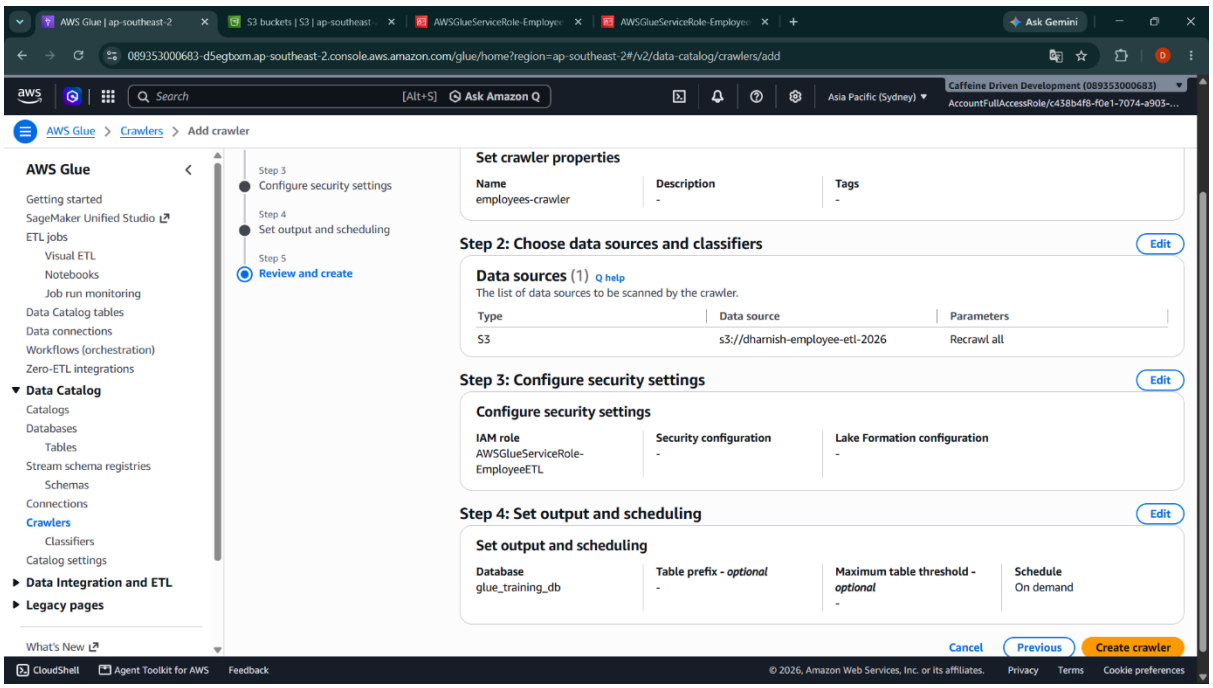
1. Open AWS Glue → Crawlers.
2. Create a crawler and enter its name.
3. Configure the data source as the S3 `raw/` location containing `employees.csv`.
4. Select the configured IAM role.
5. Set the target database to `glue_training_db`.
6. Review the configuration and create the crawler.

Explanation: A Glue Crawler scans the source dataset, infers its schema, and registers the table metadata in the Data Catalog. This avoids having to manually define the table schema for a supported CSV dataset.




One database successfully deleted
The following database is now deleted: "glue_training_db"
Configure security settings
IAM role
Existing IAM role
AWSGlueServiceRole-EmployeeETL
Create new IAM role
Update chosen IAM role
Lake Formation configuration - optional
Use Lake Formation credentials for crawling S3 data source
Security configuration - optional
Enable at-rest encryption with a security configuration.
Cancel
Previous
Next


Set output and scheduling
Output configuration
Target database
glue_training_db
Clear selection
Add database
Table name prefix - optional
Maximum table threshold - optional
Advanced options
Crawler schedule
Frequency
On demand
Cancel
Previous
Next


Set crawler properties
Name
employees-crawler
Description
Tags
Step 2: Choose data sources and classifiers
Edit
Data sources (1)
Type
Data source
Parameters
S3
s3://dharnish-employee-etl-2026
Recrawl all
Step 3: Configure security settings
Edit
Configure security settings
IAM role
AWSGlueServiceRole-EmployeeETL
Security configuration
Lake Formation configuration
Step 4: Set output and scheduling
Edit
Set output and scheduling
Database
glue_training_db
Table prefix - optional
Maximum table threshold - optional
Schedule
On demand
Cancel
Previous
Create crawler

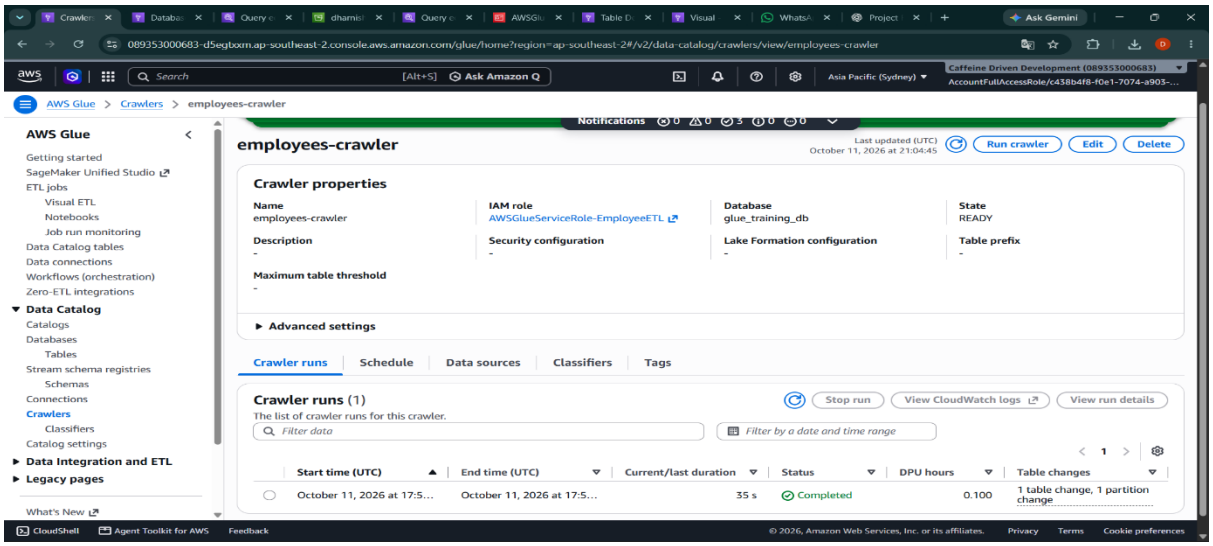
Step 5 — Run the crawler and verify its status

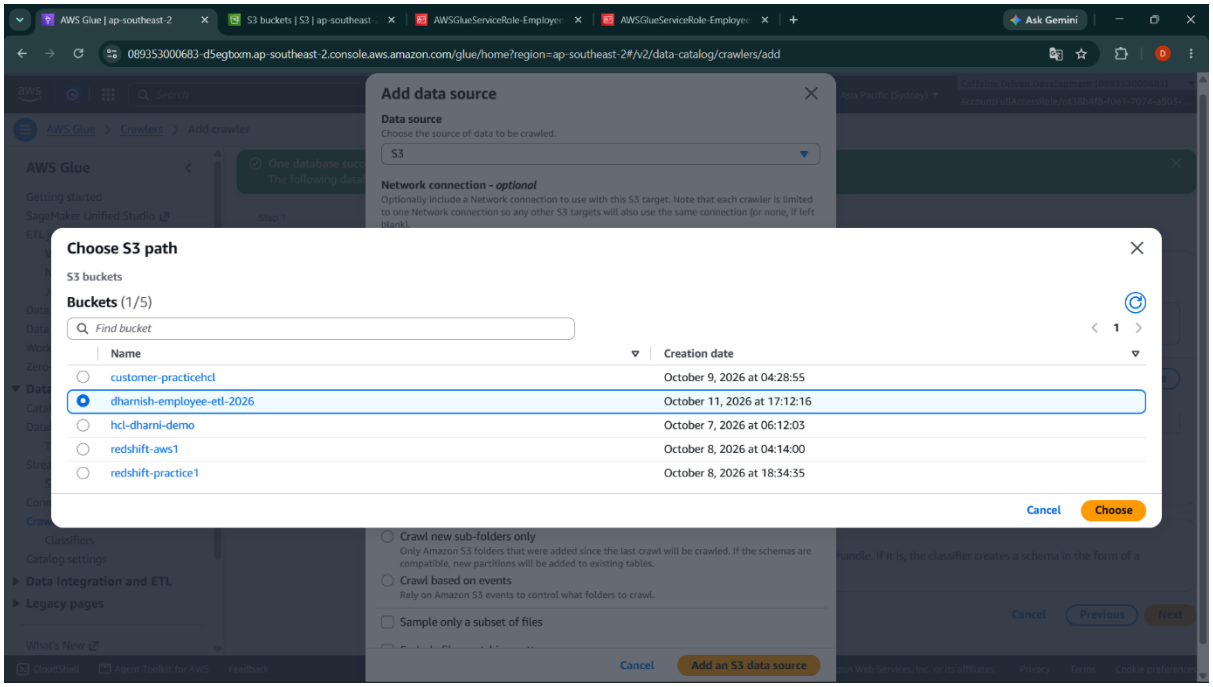
Objective: Execute the crawler and confirm that the source metadata was discovered.

Procedure:

1. Open the newly created crawler.
2. Choose Run.
3. Wait for execution to complete.
4. Check the crawler's run history and status.

Explanation: The crawler reads the configured source location and attempts to register the discovered table in the selected database. A successful run indicates that the crawler completed its execution; the table should also be checked in the Data Catalog.





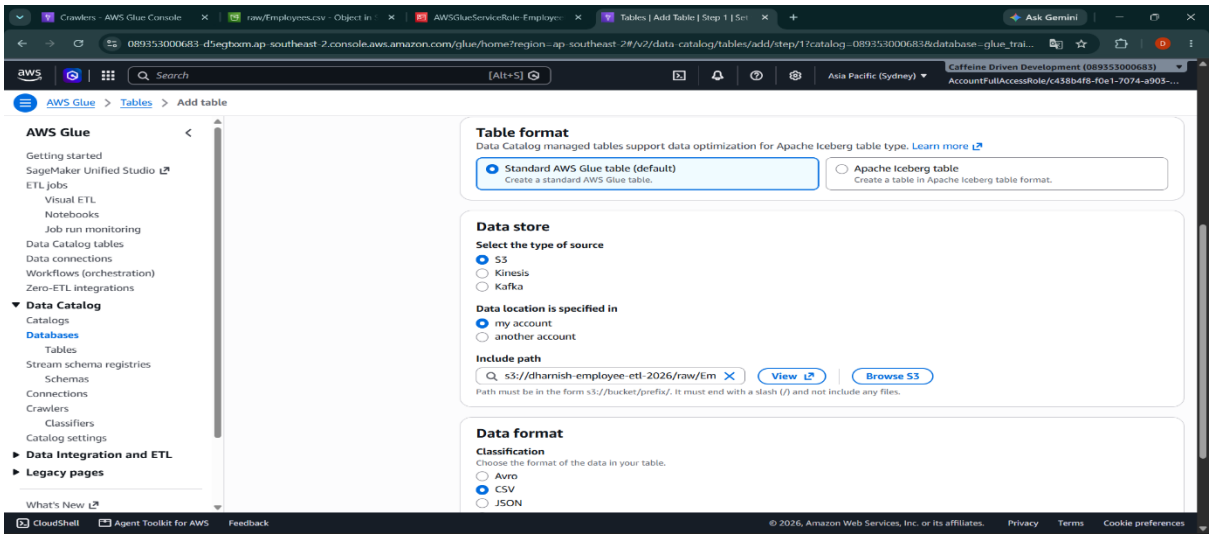
Step 6 — Verify the Glue Data Catalog table

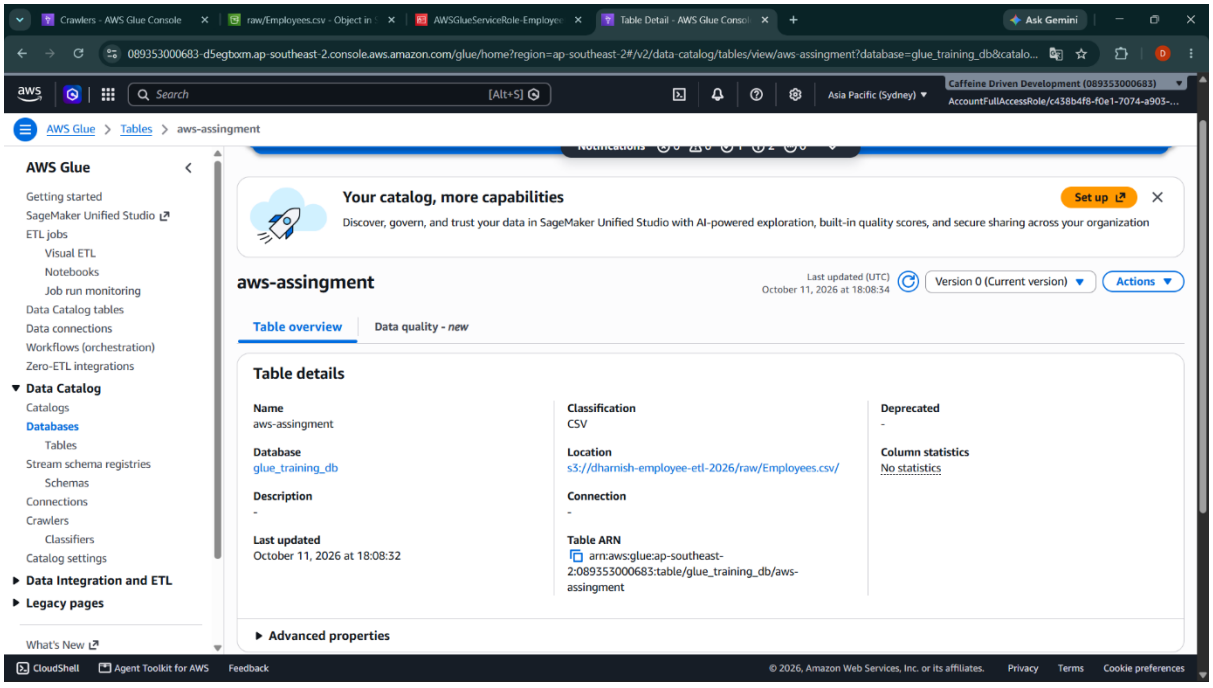
Objective: Confirm that the employee dataset is registered as a table.

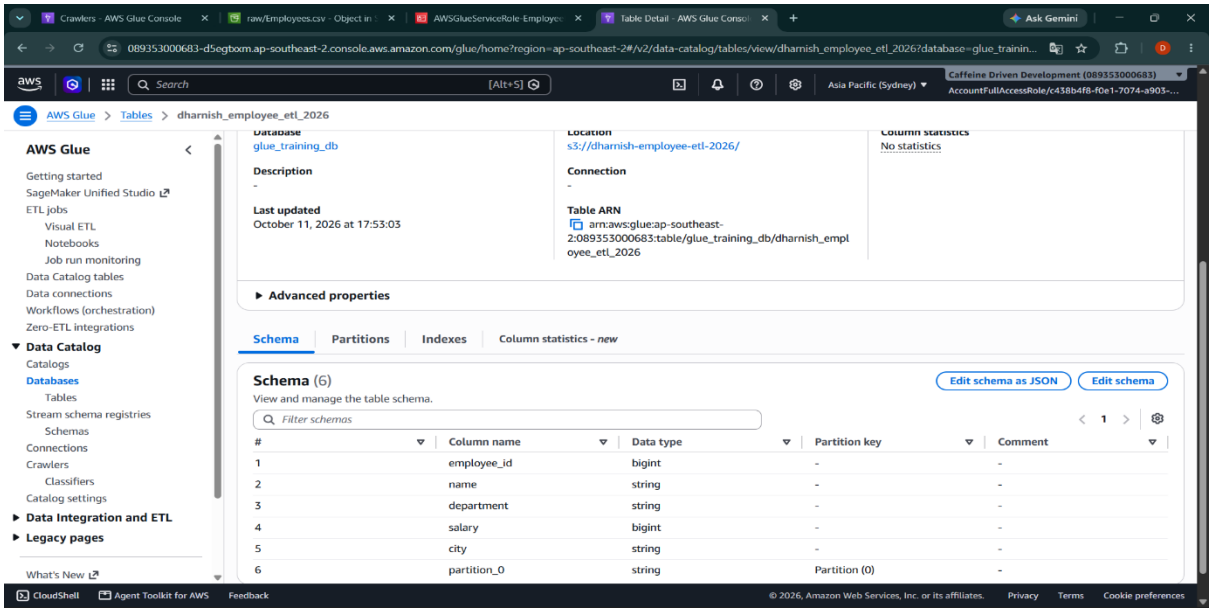
Procedure:

1. Open the Glue Data Catalog.
2. Select the `glue_training_db` database.
3. Open the `employees` table.
4. Inspect the columns and data types.

Explanation: The registered table allows AWS Glue Visual ETL and other compatible services to use the dataset's metadata as a source.







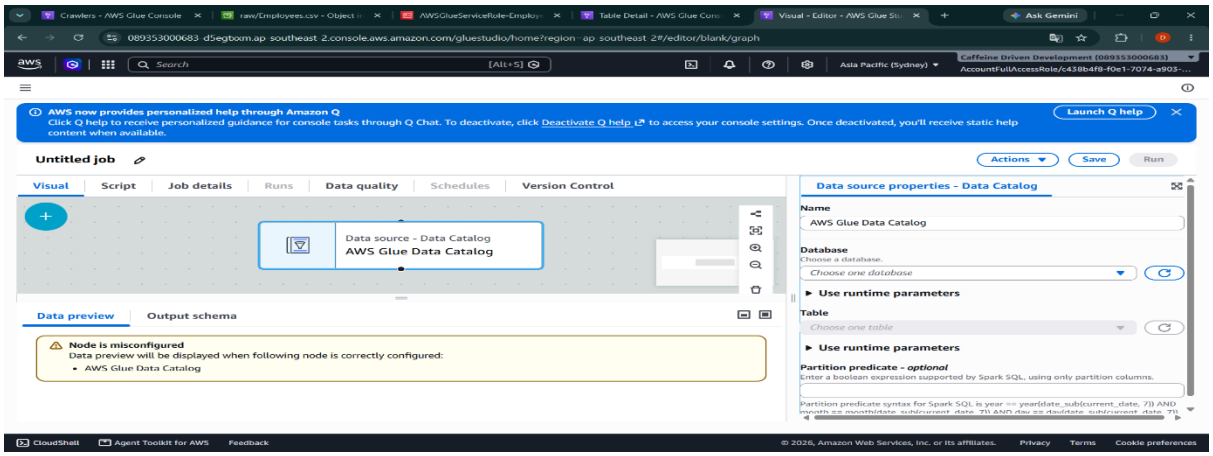
Step 7 — Create the Glue Visual ETL job

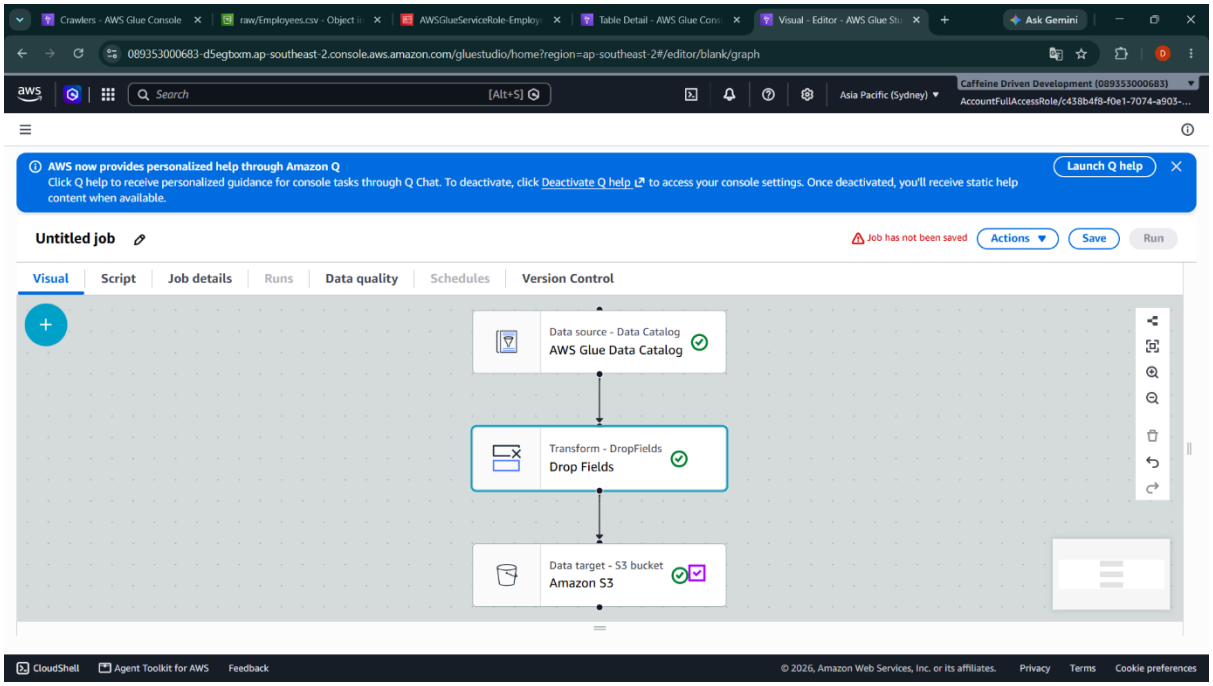
Objective: Build the transformation workflow using the registered employee table as the source.

Procedure:

1. Open AWS Glue Studio.
2. Create a new visual ETL job.
3. Add a Data Catalog source.
4. Select the `glue_training_db` database and `employees` table.
5. Add the transformation node.
6. Add an Amazon S3 target node.
7. Connect the nodes and save the job.

Explanation: Glue Visual ETL provides a graphical way to configure data processing. The source reads the registered dataset, the transformation modifies its fields, and the target writes the result to S3.





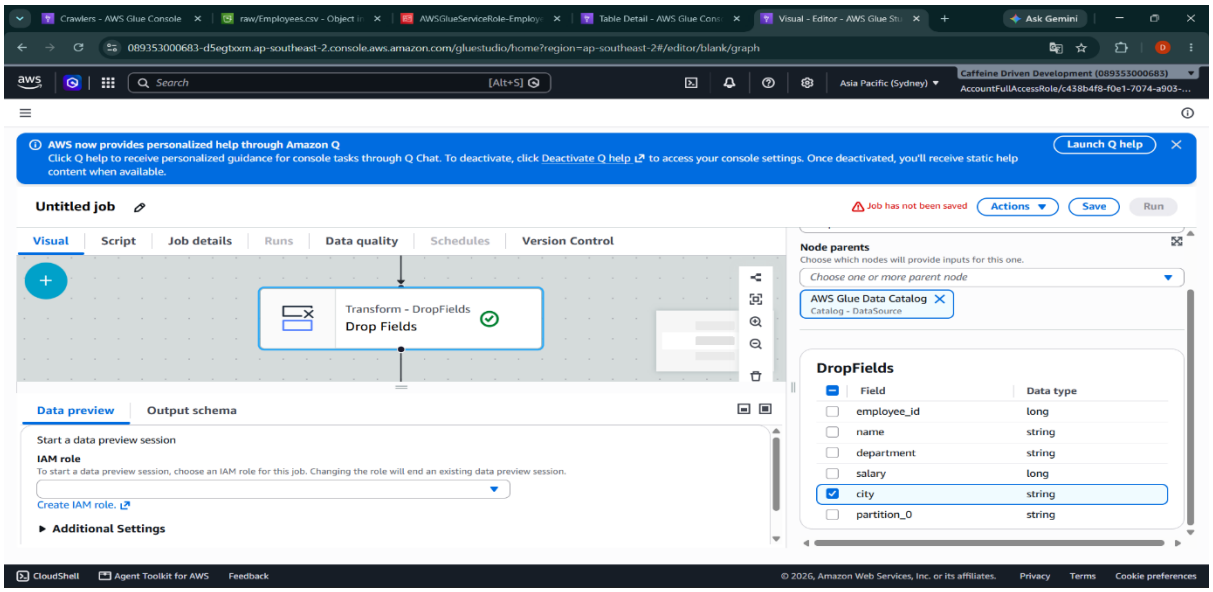
Step 8 — Apply the Drop Fields transformation

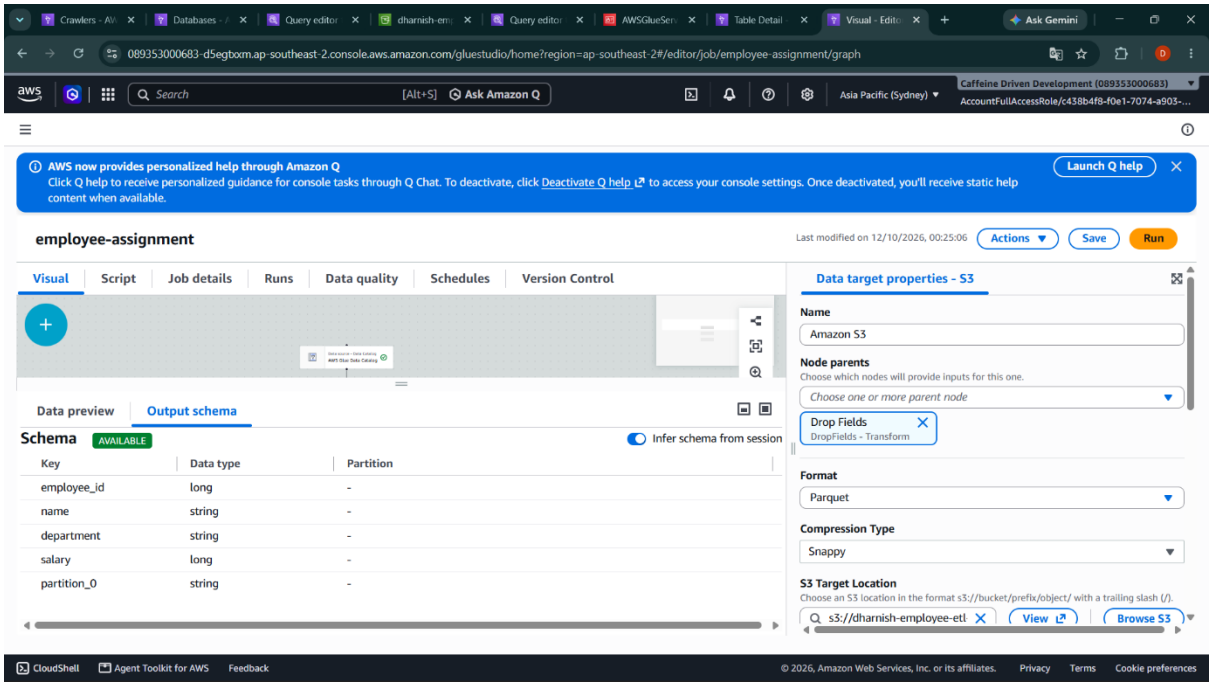
Objective: Remove the `city` field from the employee dataset.

Procedure:

1. Select the transformation node in Glue Studio.
2. Configure the Drop Fields transformation.
3. Select `city` as the field to remove.
4. Review the resulting schema.

Explanation: The Drop Fields transformation removes a selected column from the data flowing through the ETL job. In this workflow, the `city` field is removed while the remaining employee fields are retained.





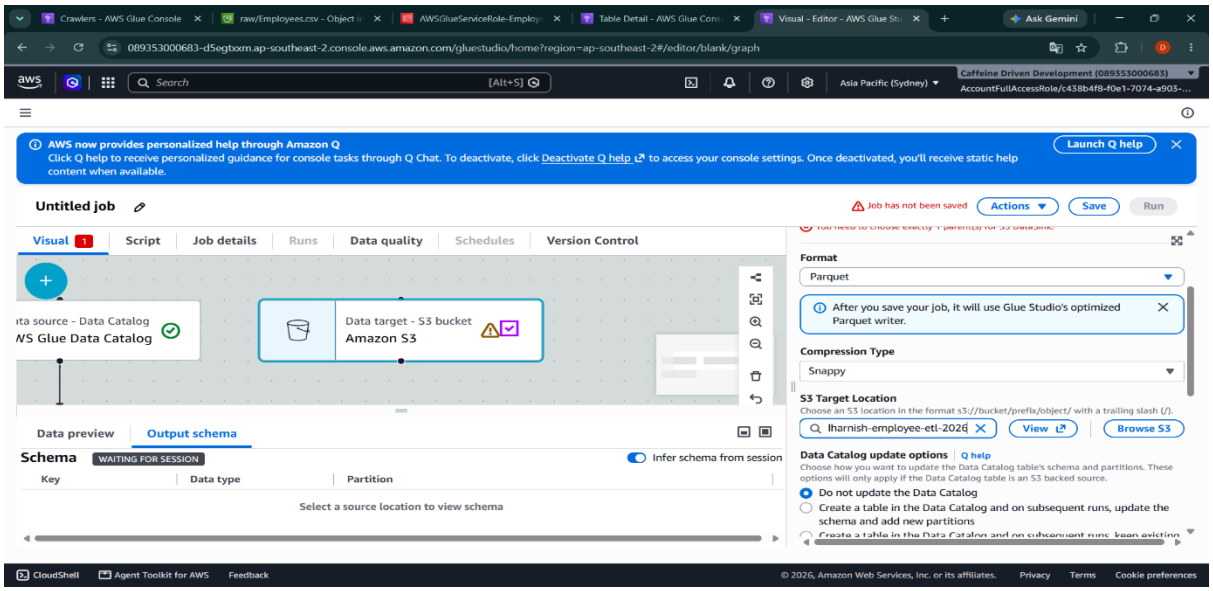
Step 9 — Configure the S3 target

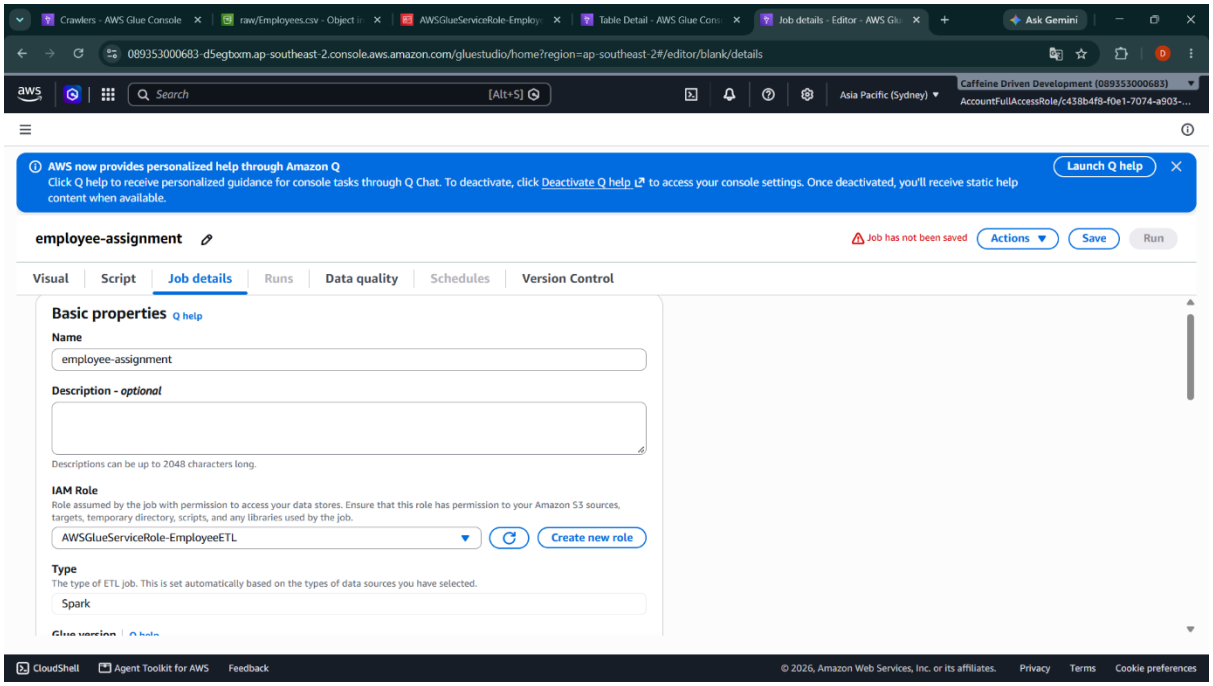
Objective: Save the transformed data in the curated S3 folder in Parquet format.

Procedure:

1. Select the S3 target node in Glue Studio.
2. Configure the target S3 path using your actual bucket and `curated/` prefix.
3. Select Parquet as the output format.
4. Review the target settings and save the job.

Explanation: The target defines where and how the transformed data will be written. Parquet is a columnar storage format commonly used for analytics because it supports efficient column-oriented reading and compression.





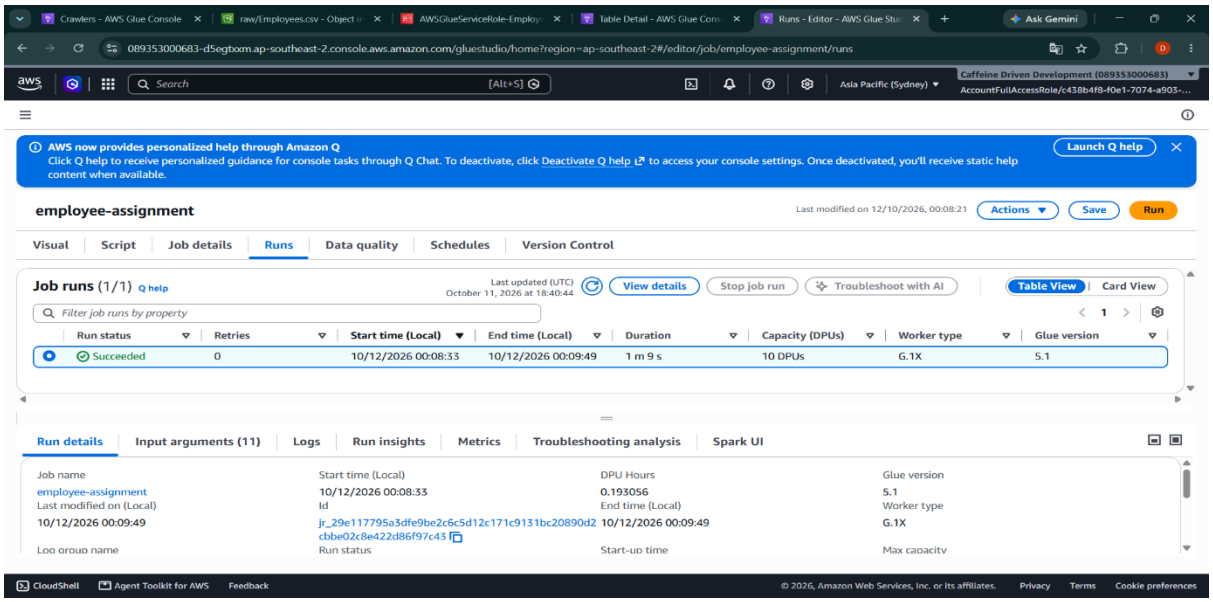
Step 10 — Run the Glue ETL job

Objective: Execute the configured data pipeline.

Procedure:

1. Save the Glue job and provide its job name and required settings.
2. Run the job.
3. Monitor the run details.
4. Verify the final status.

Explanation: During execution, the job reads the source table, applies the configured field transformation, and writes the output to the specified S3 target. A status of `Succeeded` indicates that the job run completed successfully.



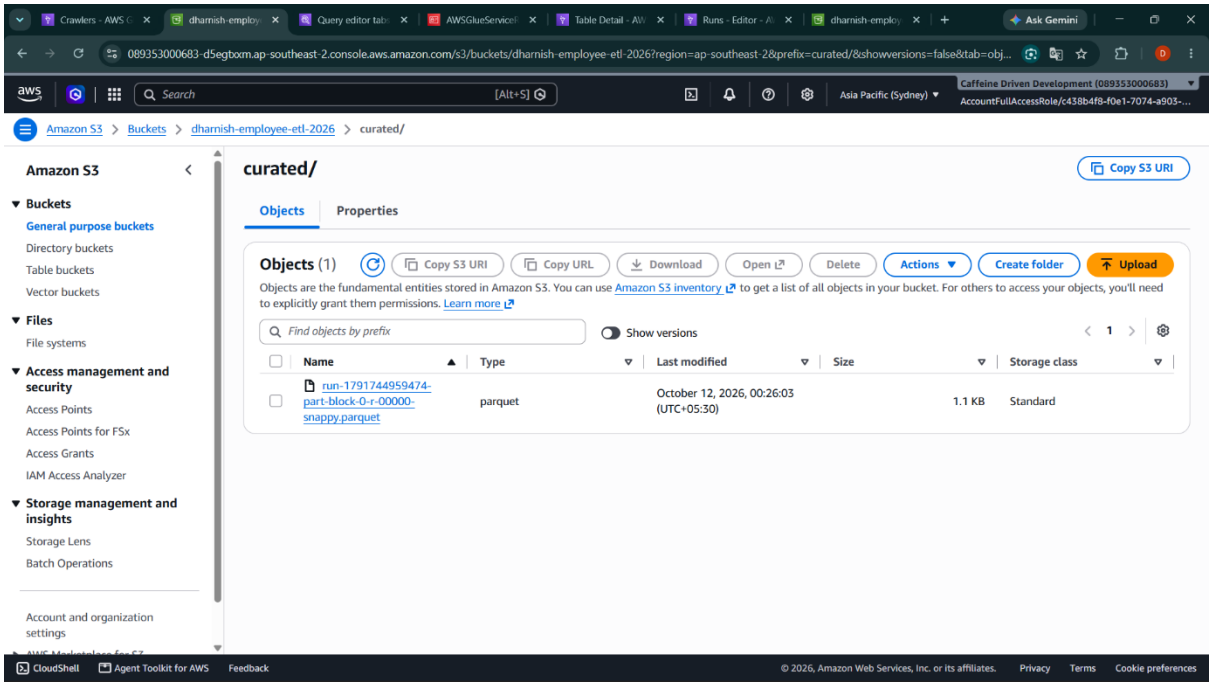
Verify the curated Parquet output in S3

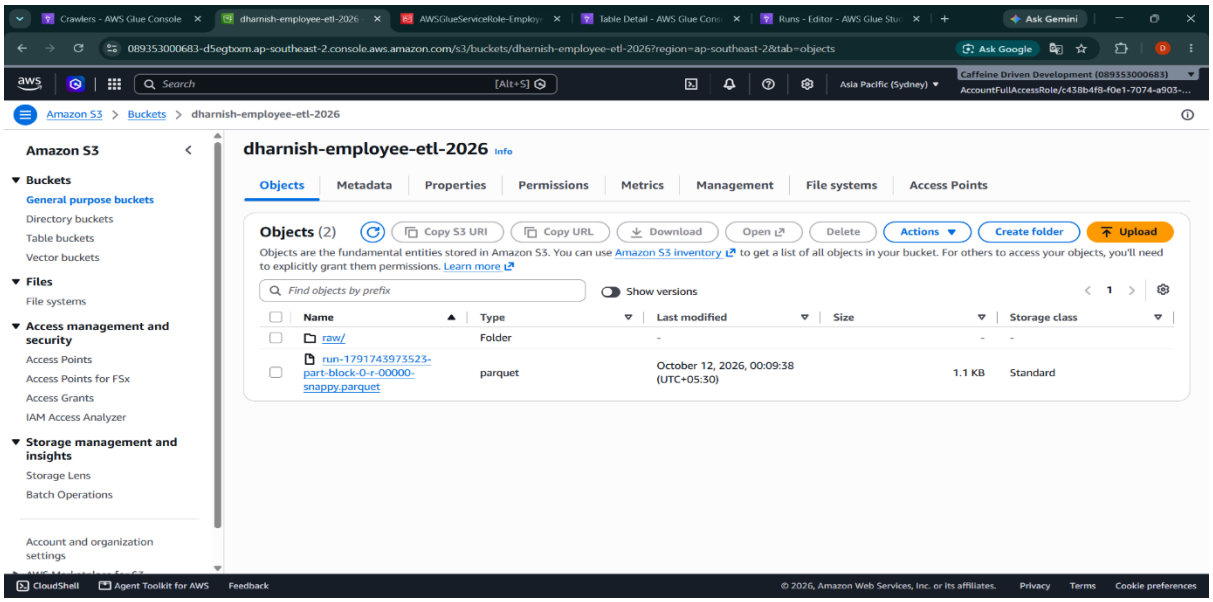
Objective: Confirm that the ETL job generated the expected output files.

Procedure:

1. Open Amazon S3.
2. Navigate to your bucket's `curated/` folder.
3. Inspect the generated output files.
4. Verify that the output is in Parquet format.

Explanation: This step validates the storage stage of the pipeline. The curated folder should contain the files generated by the Glue job. Glue may generate multiple part files and additional metadata files, depending on its configuration and execution.





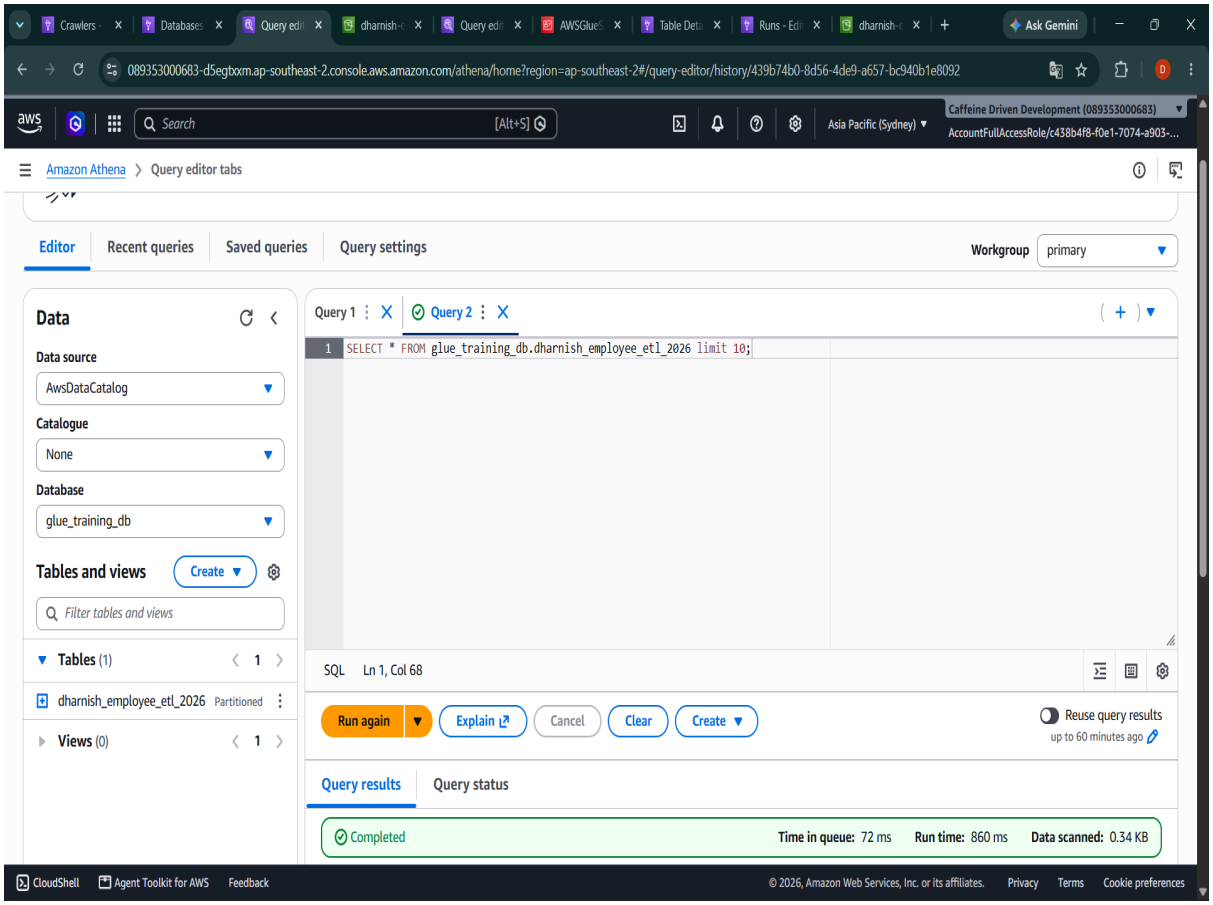
Step 12 — Validate the output using Amazon Athena

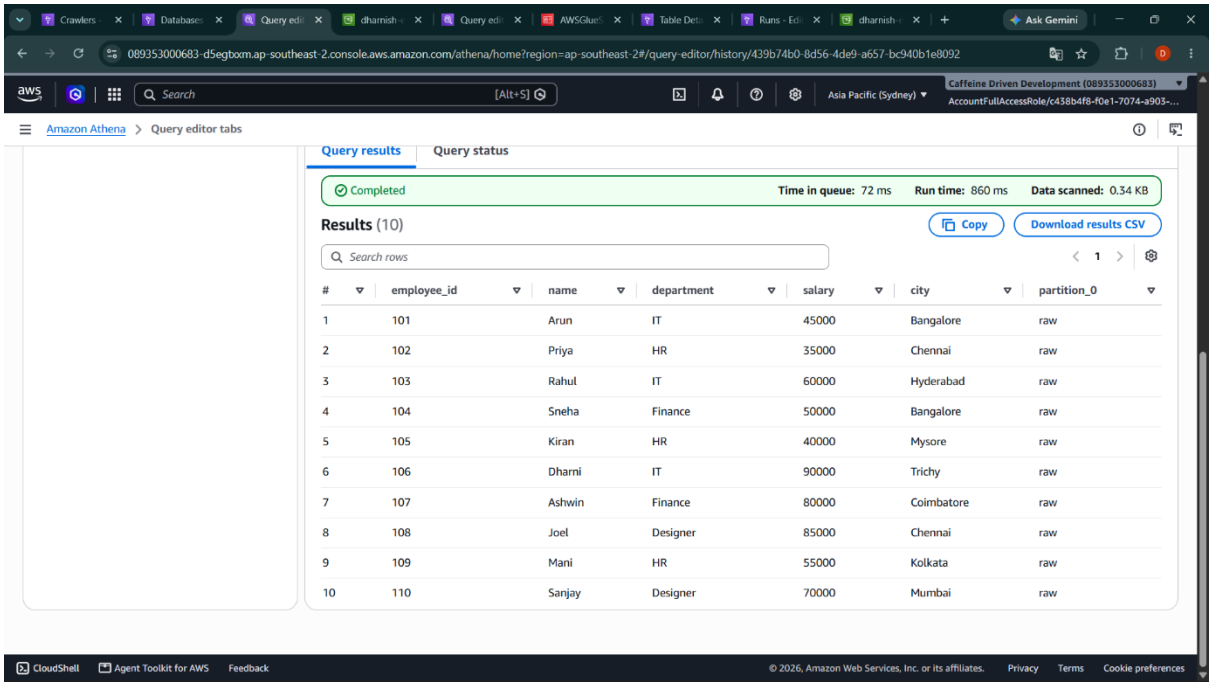
Objective: Query the processed data to verify that it can be read for analysis.

Procedure:

1. Open Amazon Athena.
2. Select the appropriate data source and catalog.
3. Select the database and table registered for the curated output, if one has been created.
4. Execute a query against that table.
5. Inspect the query results and execution status.

Explanation: Amazon Athena is a serverless query service that allows SQL queries to be executed against supported data stored in S3. It can be used to validate the processed dataset without setting up a database server.





Final workflow:

AWS Glue Visual ETL – Employee Data Processing

1. Amazon S3 – Data Storage

Created an S3 bucket named "dharnish-employee-etl-2026" to store employee data. Organized the data into "raw/" for the original CSV and "curated/" for the processed output.

2. AWS Glue Data Catalog

Created the database "glue_training_db" and cataloged the employee dataset in a table. The catalog stores metadata such as column names and data types for the ETL job.

3. AWS Glue Studio – Visual ETL

Created a Visual ETL job using three connected nodes: AWS Glue Data Catalog, Drop Fields, and Amazon S3. This workflow reads, transforms, and writes employee data.

4. Drop Fields Transformation

Configured the Drop Fields node to remove the "city" column from the employee dataset. The remaining employee fields, such as employee ID, name, department, and salary, are retained.

5. Amazon S3 Target – Parquet

Configured the S3 target to store the transformed dataset in Parquet format in the "curated/" folder. Parquet is a columnar format suitable for analytical workloads.

6. IAM Role and Permissions

Selected an IAM role to allow AWS Glue to access the source data and write the output to S3. The role must have the required permissions for the job to execute successfully.

7. Job Execution and Monitoring

Saved and ran the Glue ETL job, then checked its status in the Runs tab. The job showed Succeeded, indicating that execution completed without a reported failure.

8. Output Verification

Checked the S3 bucket for the generated Parquet file and verified the configured output location. The final validation is to confirm that the file is inside "curated/" and the output schema no longer contains "city".

Conclusion

Successfully built an AWS Glue Visual ETL pipeline to process employee data, remove an unwanted column, and store the transformed dataset in Amazon S3 for further analysis.